

PhishNet: On-Demand Browser Plugin for Email Phishing Detection Using BERT

Wangonde, Mercy Mumbi: 152063

Supervisor: Tiberius Tabulu

An Informatics and Computer Science Project Proposal Submitted to the School of Computing
and Engineering Sciences in partial fulfillment of the requirements for the award of a Degree in
Bachelor of Science in Informatics and Computer Science

School of Computing and Engineering Sciences

Strathmore University

Nairobi, Kenya

November 2025

Declaration and Approval

I declare that this project has not been submitted to Strathmore University or any other University for the award of a Degree in Bachelor of Science in Informatics and Computer Science or any other Degree. To the best of my knowledge and belief, the research contains no material previously published or written by another person except where due reference has been made in the research itself.

Student Admission Number: 152063

Sign:  _____

Date: 27/11/2025

Supervisor's Signature:

Sign:  _____

Date: 28/11/2025

Acknowledgment

I would like to extend my heartfelt gratitude to my supervisor, Mr. Tiberius Tabulu for the invaluable guidance, constructive criticism, and unwavering support throughout the development of this project. Their expertise and insights significantly shaped the direction and quality of this work. I am also profoundly thankful to the faculty members at the School of Computing and Engineering Sciences who equipped me with the necessary knowledge and skills to undertake this research. Special appreciation goes to my family and friends for their moral support, encouragement, and understanding during the demanding periods of this academic endeavor. I acknowledge all resources, both human and material, that contributed to the successful completion of this project.

Abstract

Phishing remains a serious cybersecurity threat, especially for individuals and small organizations without access to enterprise-grade protection. Attackers now use advanced tactics like AI-generated text and embedded images, which bypass traditional filters. As digital communication grows, users face increased risk from emotionally manipulative phishing attempts, highlighting the need for smarter, more accessible tools.

Current detection solutions are often expensive, require technical expertise, and lack on-demand browser integration. These systems frequently miss threats embedded in images or crafted with sophisticated language, widening the gap between user needs and available protection.

This project introduced PhishNet, a lightweight, browser-based phishing detection plugin for on-demand analysis. It integrated BERT for semantic understanding, logistic regression for classification, and OCR for image-based threats. Built with Python, FastAPI, and React.js, PhishNet flagged suspicious emails directly in the browser with minimal user input.

PhishNet's design lowered technical barriers while ensuring effective, scalable protection. By focusing on usability and responsiveness, it addressed key shortcomings in traditional systems and empowered users with real-time, AI-driven defense.

Keywords: *Browser Plugin, BERT, Logistic Regression, OCR, On-Demand Analysis.*

Table of Contents

Declaration and Approval	ii
Acknowledgment	iii
Abstract	iv
List of Figures	x
List of Tables	xii
List of Abbreviations	xiii
Chapter 1: Introduction	1
1.1 Background Information	1
1.2 Problem Statement	2
1.3 General Objective	3
1.3.1 Specific Objectives	3
1.3.2 Research Questions	3
1.4 Justifications	4
1.5 Scope and Limitations	4
1.5.1 Scope of the Project	4
1.5.2 Limitations in the Project	5
1.5.3 Delimitations in the Project	6
Chapter 2: Literature Review	8
2.1 Introduction	8
2.2 Email based Phishing Techniques and Vulnerabilities	8
2.2.1 Email-Based Phishing Techniques	8
2.2.2 Vulnerabilities in Email-Based Phishing Detection	9
2.3 Related Applications/Applications	10
2.3.1 Cofense	10
2.3.2 Barracuda	12

2.3.3	Trustifi.....	13
2.4	Gaps in the Existing Applications/ Solutions	14
2.4.1	Over-reliance on Static and Signature-Based Detection.....	14
2.4.2	High Rates of False Positives and False Negatives	14
2.4.3	Poor Resilience to Adversarial and Sophisticated Attacks	15
2.4.4	Lack of Behavioral, Context, and Interaction Analysis	15
2.5	Conceptual Framework.....	15
Chapter 3:	Development Methodology	17
3.1	Introduction.....	17
3.2	Research Paradigm.....	17
3.2.1	Data Acquisition.....	17
3.2.2	Data Preprocessing.....	18
3.2.3	Model Training.....	18
3.2.4	Model Validation and Testing	18
3.2.5	Experimental Design & Iteration	19
3.2.6	Tools Summary	19
3.3	Software Development Methodology	19
3.3.1	Product Backlog Creation & Sprint Planning.....	20
3.3.2	Sprint Design & System Architecture.....	20
3.3.3	Incremental Development (Sprint Cycles).....	21
3.3.4	Continuous Testing & Sprint Review	21
3.3.5	Sprint Retrospective & Deployment.....	21
3.4	System Analysis and Design.....	21
3.4.1	Use Case Diagram.....	21
3.4.2	Entity Relationship Diagram (ERD).....	22

3.4.3	Sequence Diagram	22
3.4.4	Wireframes / UI Mockups.....	22
3.5	System Development Tools	23
3.5.1	React.js (Frontend Framework)	23
3.5.2	FastAPI (Backend Framework)	23
3.5.3	PyCharm.....	23
3.5.4	Firebase Authentication	23
3.5.5	PostgreSQL	24
3.5.6	Tesseract OCR	24
3.5.7	Git & GitHub	24
3.5.8	Visual Studio Code	24
3.5.9	Kaggle.....	24
3.6	System Development Techniques	24
3.6.1	Natural Language Processing (NLP) with BERT	24
3.6.2	Supervised Machine Learning	24
3.6.3	OCR	25
3.7	System Deliverables.....	25
3.7.1	System Documentation	25
3.7.2	Web Plugin On-Demand Activation Module.....	25
3.7.3	AI-Based Phishing Detection Module	26
Chapter 4:	System Analysis and Design.....	27
4.1	Introduction.....	27
4.2	System Requirements.....	27
4.2.1	Functional Requirements	27
4.2.2	Non-Functional Requirements	28

4.3	System Narrative.....	29
4.4	System Design	30
4.4.1	Use Case Diagram.....	31
4.4.2	Entity Relationship Diagram (ERD).....	33
4.4.3	Sequence Diagram	35
4.4.4	Database Schema	36
4.5	System Wireframes	37
4.6	System Architecture	42
Chapter 5:	System Implementation and Testing	44
5.1	Introduction.....	44
5.2	Implementation Environment	44
5.2.1	Hardware Specifications	44
5.2.2	Software Specifications	45
5.3	Dataset Description and Preparation.....	46
5.3.1	The Nazario Phishing Corpus	46
5.3.2	The Enron Email Dataset	47
5.3.3	The SpamAssassin Public Corpus	47
5.3.4	Dataset Cleaning and Preprocessing.....	49
5.4	Description of Model Training and Evaluation	52
5.4.1	BERT Model Components	53
5.4.2	Model Evaluation Metrics.....	54
5.5	System Implementation	57
5.5.1	User Interaction Modules.....	57
5.5.2	Administrator Interaction Modules.....	60
5.6	System Testing	63

5.6.1	Authentication Testing	63
5.6.2	OCR Module Testing	64
5.6.3	BERT Model Testing.....	65
5.6.4	Plugin Integration Testing.....	67
5.7	Summary	68
5.8	Github documentation.....	69
Chapter 6:	Conclusions, Recommendations and Future Works	71
6.1	Conclusions.....	71
6.2	Recommendations.....	71
6.3	Future Works	71
References		72
Appendix		76
Appendix A1.	Gantt Chart.....	76
Appendix A2.	Turn-It-In Report.....	77

List of Figures

Figure 2.1 Cofense Vision Interface for Email Search and Quarantine.....	11
Figure 2.2 Cofense Dashboard for Real-Time Phishing Email Detection and Triage.....	11
Figure 2.3 Barracuda Email Security Service Dashboard	12
Figure 2.4 Trustifi Email Security Dashboard – Integrated Inbound and Outbound Threat Detection with Geo-Threat Visualization	14
Figure 2.5 Conceptual Framework	15
Figure 3.1 Scrum Methodology Diagram(What Is a Scrum Team - GoRetro, 2024).....	20
Figure 4.1 Use Case Diagram	31
Figure 4.2 Entity Relationship Diagram	33
Figure 4.3 Sequence Diagram.....	35
Figure 4.4 Database Schema.....	36
Figure 4.5 Browser Plugin Popup Interface.....	38
Figure 4.6 Web Dashboard Interface	39
Figure 4.7 Detailed Scan Results Interface.....	40
Figure 4.8 User Profile and Settings Interface.....	41
Figure 4.9 System Architecture Diagram.....	42
Figure 5.1 Nazario Phishing Corpus Dataset.....	47
Figure 5.2 Enron Email Dataset.....	47
Figure 5.3 SpamAssassin Dataset	48
Figure 5.4 Datasets Structure.....	49
Figure 5.5 Dataset loading functions for Enron, Nazario, and SpamAssassin.	50
Figure 5.6 Text Normalization Function for Cleaning and Standardizing Emails.....	50
Figure 5.7 Language Filtering Process.	51
Figure 5.8 Function for Generating Synthetic Legitimate Email Samples.	51
Figure 5.9 Dataset Balancing and Train, Validation, Test Splitting Process.....	52
Figure 5.10 Model Initialization and Dataset Loading Process.....	53
Figure 5.11 Model Setup, Showing Bert Loading and Class Weight Initialization.	53
Figure 5.12 Training and Validation Process Showing Epoch Progress and Metrics.	54
Figure 5.13 Final Evaluation and Saved Model Directory.	54
Figure 5.14 Confusion matrix showing model classification performance.	56

Figure 5.15 Training and validation loss and accuracy curves.	57
Figure 5.16 User Scan History.....	59
Figure 5.17 Personal Analytics Overview	59
Figure 5.18 User Export Report Page	60
Figure 5.19 Administrator Dashboard with Admin Tools Enabled.....	61
Figure 5.20 System-Wide Overview for Administrators	61
Figure 5.21 User Management Panel for Administrators	62
Figure 5.22 All Scans Admin Module.....	62
Figure 5.23 Administrator Export and Reporting Tools	63
Figure 5.24 GitHub Issues Tracking Completed Project Milestones.....	69

List of Tables

Table 5.1 Hardware Specifications	44
Table 5.2 Software Specifications.....	45
Table 5.3 Evaluation Metrics Formulas	55
Table 5.4 Final test results for the PhishNet BERT classification model.	55
Table 5.5 Authentication Testing Results.....	63
Table 5.6 OCR Module Testing Results.....	64
Table 5.7 BERT Model Testing Results	65
Table 5.8 Plugin Integration Testing Results	67

List of Abbreviations

AI - Artificial Intelligence

API - Application Programming Interface

BEC - Business Email Compromise

BERT - Bidirectional Encoder Representations from Transformers

CSS - Cascading Style Sheets

CSV - Comma-Separated Values

DLP - Data Loss Prevention

ERD – Entity Relationship Diagram

HTML - HyperText Markup Language

HTTPS - HyperText Transfer Protocol Secure

ID - Identifier

JS - JavaScript

JSON - JavaScript Object Notation

NLP - Natural Language Processing

OCR - Optical Character Recognition

OOAD - Object-Oriented Analysis and Design

PDF - Portable Document Format

QR - Quick Response

REST - Representational State Transfer

SIEM - Security Information and Event Management

SOC - Security Operations Center

SQL - Structured Query Language

UI - User Interface

URL - Uniform Resource Locator

UX - User Experience

YARA - Yet Another Recursive Acronym

Chapter 1: Introduction

1.1 Background Information

Phishing attacks continue to be a significant cybersecurity threat targeting individuals, businesses, and institutions. These attacks exploit users' trust by impersonating legitimate entities to obtain sensitive information such as usernames, passwords, credit card numbers, and other personal data. As email and digital communication dominate both professional and personal interactions, phishing attacks have become increasingly frequent and damaging. According to the 2023 Verizon Data Breach Investigations Report, phishing accounted for 36% of all data breaches (Verizon, 2023).

Despite growing awareness, many phishing detection and filtering solutions are not well suited for individual users or small teams. Most existing tools are enterprise-oriented, vendor-based platforms offered as part of broader cybersecurity suites that operate on freemium or subscription models. Critical features such as real-time filtering or automatic blocking are often restricted to paid tiers, making these solutions inaccessible or difficult to customize for individuals or resource-constrained organizations. Furthermore, many tools require manual user engagement, such as copying suspicious email content into external scanners, which increases the risk of human error (Korkmaz, 2023). Over 90% of phishing campaigns rely on social engineering tactics, highlighting the need for detection models that operate passively and on-demand without relying on user intervention (Vijayalakshmi et al., 2020).

Individuals and small businesses remain particularly vulnerable due to limited cybersecurity training and infrastructure. In 2022, 90% of small businesses reported phishing attempts, yet only half had trained their employees to recognize such threats (Verizon, 2023). This underscores the need for intuitive, accessible, and affordable phishing detection tools tailored to non-enterprise users.

Natural Language Processing (NLP) has emerged as a powerful technique for phishing detection by analyzing email language for deceptive cues such as manipulation, urgency, and psychological triggers (Kumrashan Indranil Iyer, 2025). NLP can identify subtle linguistic patterns that traditional keyword-based filters miss. Additionally, Optical Character Recognition (OCR) has become increasingly useful for extracting characters from images and embedded content, enabling

detection of visually hidden phishing attempts. When combined and deployed in browser extensions or on-demand scanning tools, NLP and OCR significantly enhance real-time user protection(Korkmaz, 2023).

The adoption of NLP and OCR in phishing detection systems has significantly strengthened digital security by enabling real-time, automated analysis of email content. These technologies reduce reliance on human vigilance and help prevent data breaches by identifying phishing attempts before they reach end-users. As a result, they enhance organizational resilience against social engineering attacks and minimize data exfiltration risks(Proofpoint, 2024).

1.2 Problem Statement

Phishing attacks have evolved into highly sophisticated operations, increasingly leveraging artificial intelligence (AI) to craft convincing fraudulent communications. These attacks exploit both human vulnerabilities and technological gaps, making them more difficult to detect and prevent. In 2023, phishing accounted for 36% of all data breaches, emphasizing its prominence as a leading cybersecurity threat (Verizon, 2023).The integration of AI has allowed attackers to automate and scale their operations, producing emails that are grammatically flawless, contextually relevant, and highly persuasive (TechRadar, 2025).

Current anti-phishing solutions are predominantly designed for enterprise environments and often require substantial technical expertise and resources. Key features such as real-time detection and advanced filtering are typically reserved for premium, paid tiers, leaving individual users and small businesses with inadequate protection(SentinelOne, 2025).Many solutions also require manual intervention for example, copying and pasting suspicious content into external tools which slows down detection and increases the likelihood of user error (Korkmaz, 2023).

To address these limitations, this project proposes PhishNet, an on-demand phishing detection and filtering system designed for deployment as a browser plugin. The model integrates BERT for semantic language understanding with Logistic Regression for efficient classification. PhishNet activates only upon user request, minimizing background resource usage while providing dynamic, context-aware analysis of email content directly within the browser. To address attacks hidden within images, the system also incorporates OCR to extract and analyze text embedded in screenshots or image-based email content. The model is built and evaluated within a browser

extension environment to assess its detection accuracy, responsiveness, and real-world usability in identifying phishing attempts at the point of interaction.

1.3 General Objective

To develop a browser-based phishing detection and filtering system that integrates BERT for semantic language analysis and classification with OCR for image-based text extraction, deployed as an on-demand web extension to identify and flag phishing content in real time.

1.3.1 Specific Objectives

- i. To investigate common phishing techniques and vulnerabilities that affect users through email-based attacks.
- ii. To analyze BERT-based NLP models and OCR technologies that could be used for effective phishing email detection and filtering.
- iii. To review related phishing detection works, analyzing their methods and identifying challenges in usability, deployment, and accessibility.
- iv. To develop an email filtering model that combines BERT for feature extraction and classification to detect and flag phishing emails.
- v. To test the phishing detection model in a web-based plugin, evaluating its accuracy and real-time performance.

1.3.2 Research Questions

- i. What are the most prevalent phishing techniques and email-based vulnerabilities currently exploited to compromise user security?
- ii. How can BERT-based NLP and OCR technologies be effectively integrated into a browser plugin to enhance phishing email detection?
- iii. What are the core methods used by existing phishing detection tools, and what limitations do they face in terms of usability, deployment, and accessibility?
- iv. How can a combined BERT-based model be designed to accurately detect and flag phishing emails?

- v. How accurate and efficient is the proposed phishing detection model when deployed as a real-time web browser plugin?

1.4 Justifications

As digital platforms increasingly shape daily life, cybersecurity must evolve from specialized infrastructure into accessible, embedded tools that protect users seamlessly. Web browsers have become central to personal and professional communication, especially email, making them a natural venue for real-time security integration. However, phishing detection is still often treated as a backend enterprise concern, with limited focus on empowering everyday users directly at the point of interaction.

Most free phishing detection tools operate within rigid vendor ecosystems or enterprise-focused platforms, offering limited functionality and requiring technical knowledge or manual engagement. These limitations make it difficult for individuals, students, and small businesses to protect themselves effectively especially when phishing emails bypass traditional keyword filters using deceptive formatting or embedded images.

This project addresses those gaps by leveraging browser plugins to deliver intelligent, lightweight phishing detection. PhishNet integrates BERT for natural language understanding and Logistic Regression for classification, alongside OCR to process embedded or image-based text. Its on-demand architecture ensures users can scan email content only when needed, without draining system resources.

By embedding these AI-driven techniques into a user-friendly browser plugin, the project enhances on-demand threat visibility and raises awareness around secure digital behavior, helping users of all backgrounds build stronger defenses without disrupting daily workflows.

1.5 Scope and Limitations

1.5.1 Scope of the Project

This project involves the design, development, and testing of PhishNet, an AI-driven phishing detection and filtering model implemented as a Google Chrome browser plugin. It aims to provide users with a lightweight, on-demand tool that analyzes and flags phishing attempts in emails viewed directly within the browser. The plugin supports both textual and image-based content by

integrating NLP and OCR. BERT is employed to interpret semantic patterns in email text, and to classify message as legitimate or phishing based on extracted features. OCR enables detection of text hidden within images such as screenshots or obfuscated content, enhancing the plugin's ability to detect phishing attempts beyond standard text-based filters.

The testing phase primarily targets image-based phishing inputs, such as screenshots of emails. The model is evaluated by its ability to extract text from these images using OCR, process the content with BERT, and assess phishing likelihood. Key evaluation metrics include detection accuracy, processing speed, and overall usability within the browser. The project delivers a working Chrome extension that allows users to initiate phishing detection on demand, without requiring constant background monitoring or complex setup. The system is self-contained within the browser and does not depend on external tools or servers. Final deliverables include the phishing detection model, the integrated plugin interface, a structured demonstration using image-based phishing samples, and the ability to detect and analyze emails opened in the browser.

The project boundaries are clearly defined. It does not include continuous inbox scanning, real-time protection outside the browser, or phishing detection from dynamic web forms, or non-email content. The plugin is tested only on Google Chrome and supports only English-language inputs. Furthermore, it does not address broader cybersecurity threats such as malware, ransomware, or spam filtering. These exclusions maintain a focused scope that ensures technical feasibility and alignment with the project's core objectives.

1.5.2 Limitations in the Project

One major limitation of this project is the dependency on high-quality training data. The accuracy of the phishing detection model, particularly the BERT-based NLP is only as strong as the dataset it is trained on. If the dataset lacks balance, contains mislabeled samples, or fails to represent realistic phishing scenarios, the model's performance in real-world environments may be compromised.

Another limitation lies in the technical constraints of web browser environments, especially when deploying AI models through browser plugins. Browsers such as Google Chrome impose restrictions on processing power, memory usage, and background execution, which may affect the

plugin’s ability to perform real-time analysis effectively. Complex models may slow down the user experience or fail to run altogether on lower-end devices.

There is also a limitation related to the variability in the clarity and formatting of input images. Since part of the detection process relies on OCR to extract text from email screenshots or embedded images, poor image resolution, unusual fonts, or low contrast may hinder OCR accuracy, which in turn affects overall detection reliability.

Publishing the plugin in a browser environment presents practical challenges. During development, the extension is tested in Chrome's developer mode, which does not fully represent a production deployment through the Chrome Web Store. As a result, factors such as security permissions, user installation experience, and automated updates cannot be fully simulated.

1.5.3 Delimitations in the Project

The project uses publicly available and academically recognized phishing datasets that are well-labeled, diverse, and representative of both textual and image-based phishing content. Additional data augmentation techniques may also be applied to simulate realistic variations in phishing attempts. While perfect accuracy may not be achievable, careful dataset selection and preprocessing help build a model that generalizes well within the intended browser plugin use case.

To mitigate computational constraints, the model architecture is deliberately kept lightweight by pairing BERT for feature extraction, balancing detection capability with efficiency. Additionally, the plugin operates in an on-demand mode rather than continuous monitoring, ensuring it runs only when initiated by the user. This approach helps manage browser limitations while maintaining responsive detection performance.

The OCR component is tested using a curated set of clear and common image formats typical of browser-rendered content. Basic pre-processing steps such as grayscale conversion, contrast adjustment, and image scaling are integrated to enhance text clarity before OCR extraction. While the model may still struggle with highly distorted images, these enhancements aim to maximize accuracy within reasonable input quality expectations.

The plugin is thoroughly tested in developer mode with attention to compliance with Chrome's extension guidelines. Documentation is prepared to streamline the future process of submitting the plugin for official publication. Although developer mode has its constraints, it provides a realistic sandbox for evaluating plugin behavior and user interaction prior to formal release.

Chapter 2: Literature Review

2.1 Introduction

This chapter delves into the challenges users face with existing phishing detection systems, examines current solutions and the algorithms they employ, and identifies gaps in the field. It also explores email-based phishing techniques and vulnerabilities, providing context for how attackers exploit human and technical weaknesses. The chapter concludes with the conceptual framework guiding the development of PhishNet, a browser-based, BERT-powered detection plugin designed to enhance user security and awareness.

2.2 Email based Phishing Techniques and Vulnerabilities

2.2.1 Email-Based Phishing Techniques

This subsection focuses on the techniques employed by attackers to craft and deliver phishing emails that evade detection and deceive users. Phishing continues to be one of the most persistent cybersecurity threats, with attackers adopting increasingly sophisticated techniques that evade legacy detection tools. Traditional defenses such as URL blacklists, pattern-matching filters, and sender reputation systems have proven inadequate in detecting modern phishing attacks powered by AI-generated content and visually deceptive formats(Optimized Deep Learning Based Phishing Email Detection Using BERT and Hill Climbing Algorithm | SpringerLink, 2025).

A key trend in modern phishing is the use of highly personalized and multimedia-embedded emails such as spear phishing and quishing. Spear phishing targets individuals using personalized information such as their name, job title, or previous interactions to create convincing messages. These emails often use psychological cues like urgency or authority to manipulate recipients. Similarly, quishing leverages QR codes embedded in email images or attachments to redirect users to credential-stealing websites (Lookout, 2022).

Recent advancements in phishing detection focus on integrating deep learning and NLP models. BERT, a transformer-based language model, demonstrates exceptional performance in analyzing semantic content and detecting textual deception with F1 scores above 0.99 (Deep Learning in Security, 2023). Logistic Regression, while less complex, is frequently used for structured feature classification and is often paired with BERT for real-time inference efficiency (Comparative

Analysis of Machine Learning Algorithms for Email Phishing Detection Using TF-IDF, Word2Vec, and BERT - ScienceDirect, 2024).

Visual deception has also become a key tactic. Attackers increasingly embed phishing messages inside images to avoid text-based filters. To counter this, modern systems now integrate OCR, allowing detection of threats hidden in HTML images, email banners, or logos.

2.2.2 Vulnerabilities in Email-Based Phishing Detection

This subsection outlines the vulnerabilities exploited by phishing attacks and the limitations that affect users and systems trying to detect them. Despite advances in detection technology, phishing attacks continue to exploit human behavior and systemic weaknesses. Users remain highly vulnerable to deception through emotional manipulation, especially when emails appear to come from trusted sources. Fear, urgency, and authority remain central tactics that increase the likelihood of users clicking on malicious content (Labs, 2023).

The rise of AI-generated emails further complicates detection. A study by the Harvard Kennedy School found that phishing messages created by large language models had a 54% success rate in deceiving users comparable to expertly crafted human attempts (Harvard Kennedy School, 2024). These messages mimic grammar, tone, and urgency so effectively that they are nearly indistinguishable from legitimate communications.

In enterprise settings, Bring Your Own Device (BYOD) policies exacerbate this risk by extending exposure to personal devices that may lack enterprise-grade protection. Many consumer-grade email apps do not effectively detect advanced threats like AI-crafted phishing or QR code abuse (Lookout, 2022).

Usability remains a challenge. Users often find browser plugins or phishing detection tools too complex to install or understand. Real-time plugins, while technically effective may be distrusted by users or ignored entirely if alerts are poorly designed or overly frequent. These usability gaps leave both individual users and organizations exposed to sophisticated phishing threats.

2.3 Related Applications/Applications

2.3.1 Cofense

Cofense is an email security and incident response platform that helps organizations detect and respond to phishing threats. It works by analyzing user-reported emails through smart clustering algorithms, noise reduction filters, and customizable Yet Another Recursive Acronym (YARA) rules. It integrates with tools like SIEMs and sandboxes to automate threat responses, such as quarantining emails or blocking URLs. Cofense also uses reporter reputation scoring to prioritize more accurate reports and provides dashboards and automated workflows to streamline SOC operations(Cofense Email Security, 2024) . Its core users are enterprise SOC teams across sectors like finance, healthcare, and government, who rely on it to reduce phishing risk and false positives.

Users of Cofense Triage may encounter several practical challenges. First, configuring detection rules using YARA or custom logic can be complex and time-consuming, requiring a high level of expertise that smaller teams may lack. This increases the risk of misconfigured rules or blind spots in detection. Second, because the platform heavily depends on user-reported emails, phishing attempts that go unnoticed by users can slip through undetected, despite features like "Who Else" that attempt to identify unreported threats. Third, while Cofense supports integrations with other tools, setting them up and maintaining them especially across cloud, on-prem, and hybrid environments can introduce technical burdens. Additionally, the system's limited real-time capabilities mean it focuses on reactive rather than proactive defense, which can delay threat mitigation and leave systems vulnerable to fast-moving attacks.

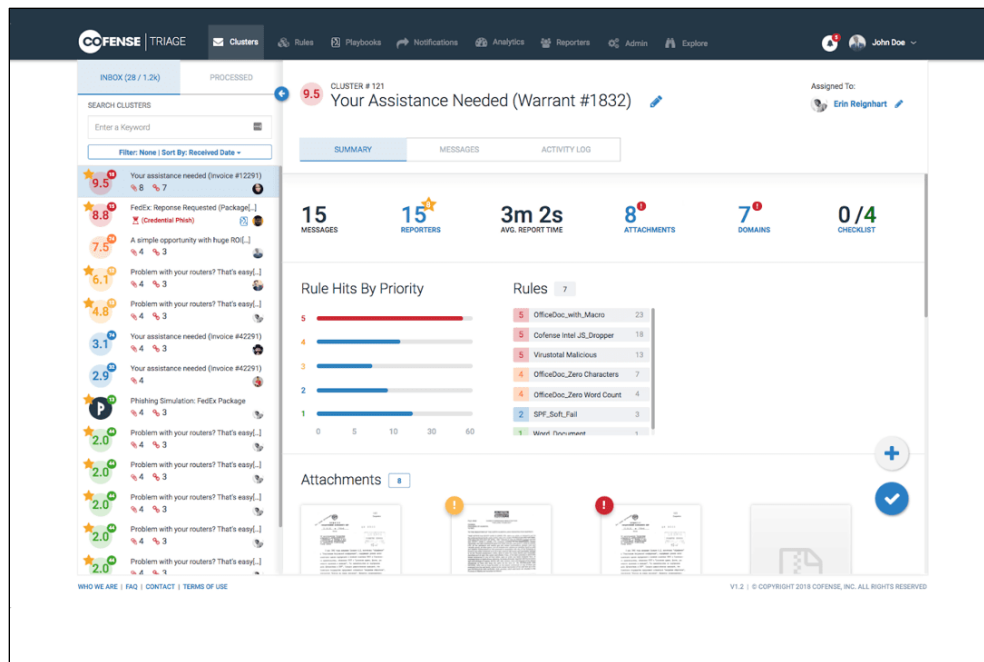


Figure 2.1 Cofense Vision Interface for Email Search and Quarantine

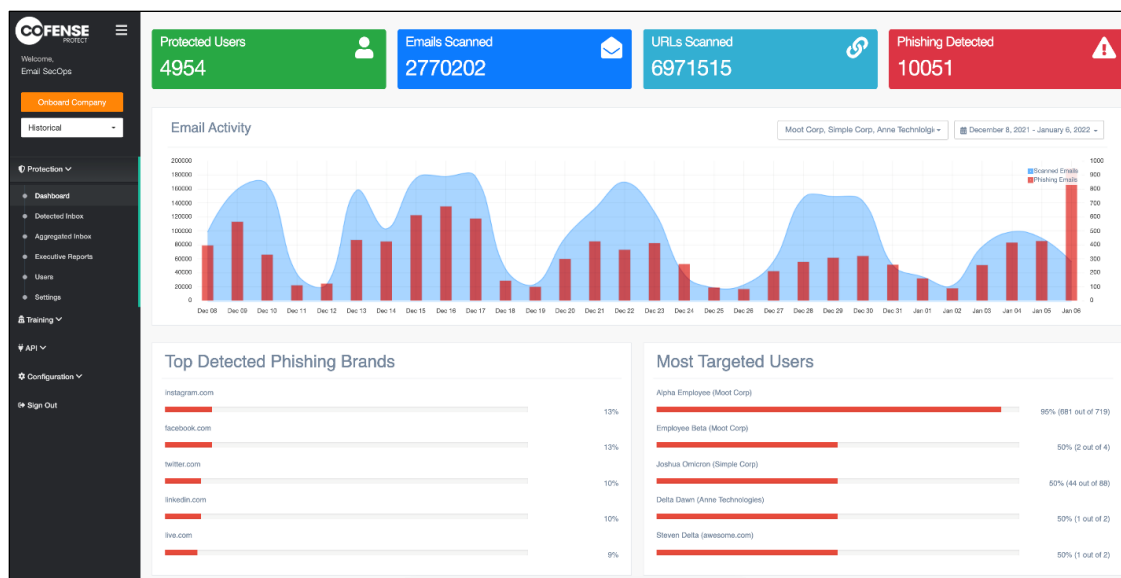


Figure 2.2 Cofense Dashboard for Real-Time Phishing Email Detection and Triage

2.3.2 Barracuda

Barracuda Email Protection uses a multi-layered approach combining gateway filtering and inbox defense, powered by AI, machine learning, and real-time threat intelligence. Emails are first filtered at the gateway through virus scanning, spam scoring, URL protection, and content reputation checks using Barracuda Central's threat network. Suspicious attachments are analyzed in a sandbox to detect zero-day threats. At the inbox level, API-based tools detect impersonation and account takeovers using behavioral analytics, communication history, and natural language processing. Features like link rewriting, QR code scanning, outbound encryption (AES-256), and cloud-based email continuity further strengthen protection (Barracuda Networks, 2023).

Users may encounter some issues with Barracuda's system. Deploying both the gateway and inbox API layers across hybrid or cloud environments can be complex, requiring correct MX record setup and API configuration. While AI reduces false positives, maintaining rules and tuning spam thresholds requires ongoing admin effort. Real-time updates and sandboxing can slow message delivery, and although effective against known threats, the system may struggle to catch fast-evolving phishing attacks that exploit deepfake content or new social engineering tactics before detection models adapt.

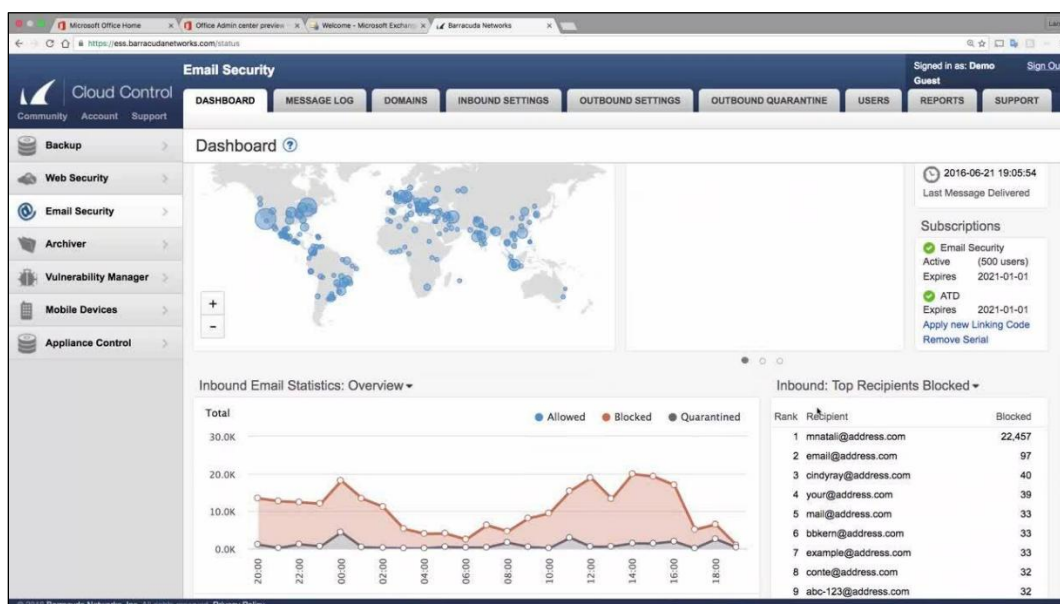


Figure 2.3 Barracuda Email Security Service Dashboard

2.3.3 Trustifi

Trustifi is a cloud-based email security platform designed to protect both inbound and outbound messaging using multi-layered AI-driven defenses. The Inbound Shield scans incoming emails through content analysis, URL and attachment inspection, and sophisticated heuristics to detect impersonation, spoofing, phishing, ransomware, spam, and graymail all backed by machine learning models and real-time threat intelligence. The Outbound Shield adds customizable rules for DLP and compliance, applying AES-256 encryption automatically based on defined policies for sensitive content. Trustifi also features Account Takeover Protection, which uses behavioral analytics on user metadata to identify anomalies and block compromised accounts in real-time. Additional services include managed detection and response, cloud archiving with e-discovery capabilities, and integrated phishing awareness training via simulation exercises. The system integrates via Outlook, Gmail, and APIs, making deployment seamless without requiring MX record changes (Trustifi, 2024).

Despite its comprehensive coverage, Trustifi poses several adoption challenges. First, the platform's extensive feature set ranging from DLP and encryption to behavior analytics may overwhelm smaller IT teams, requiring significant configuration and tuning to align policies with organizational needs. Second, the reliance on AI and behavioral anomaly detection, while reducing manual oversight, may lead to false positives or negatives that require regular monitoring to adjust thresholds and rules. Third, while no MX record changes are needed, integrating inbox plugins across Outlook, Gmail, and APIs can still introduce compatibility issues or performance overhead, particularly in hybrid or mobile environments. Finally, Trustifi is strong on protecting known threats and regulated content but may face gaps against rapidly evolving phishing campaigns or deepfake-enabled social engineering before its models detect new patterns.

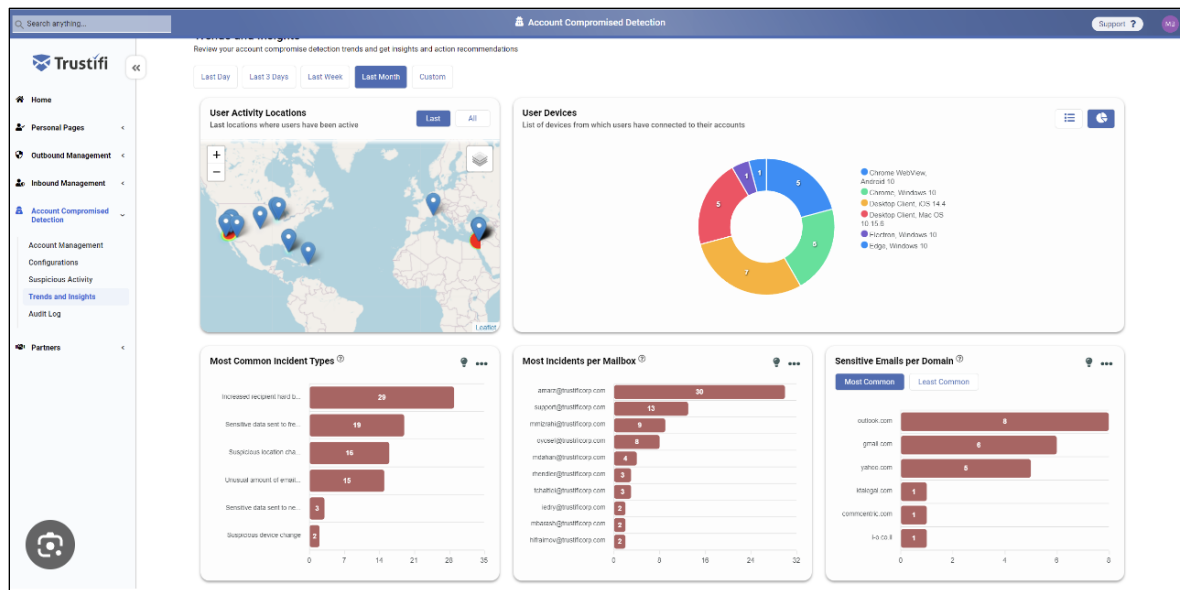


Figure 2.4 Trustifi Email Security Dashboard – Integrated Inbound and Outbound Threat Detection with Geo-Threat Visualization

2.4 Gaps in the Existing Applications/ Solutions

A thorough evaluation of the reviewed systems (Cofense, Barracuda Email Protection, and Trustifi) reveals persistent gaps in email security and phishing detection. The key points outlining these gaps are included below:

2.4.1 Over-reliance on Static and Signature-Based Detection

Most email security platforms still heavily depend on malware signatures, blacklists, and fixed rules. As noted in a recent industry analysis, such static defenses miss over 53% of modern threats, especially evasive Business Email Compromise (BEC) and AI-generated phishing that bypass these mechanisms (The Three Main Security Gaps of Email Gateway Solutions | Votiro, 2021). Cyber adversaries now craft polymorphic emails and use fresh domains to slip past outdated filters and sandboxes before updates can occur.

2.4.2 High Rates of False Positives and False Negatives

Legacy detection models that rely on rules and heuristics often struggle with accuracy. Studies indicate that standard email defenses block legitimate communications (false positives) and miss sophisticated attacks (false negatives), causing operational inefficiencies and elevated risk. For

example, Cofense and Trustifi users commonly report excessive quarantine emails or important messages misclassified, and teams spend up to 25% of their time reviewing such errors (The G2 on Trustifi Email Security, 2025).

2.4.3 Poor Resilience to Adversarial and Sophisticated Attacks

Recent research shows that even deep-learning spam filters can be deceived through adversarial manipulation at the word and sentence level (Hotoğlu et al., 2025). Techniques like intentional typo-squatting, semantic poisoning, and stealthy embedding of malicious payloads remain largely undetected by current defenses, exposing gaps in real-world protection.

2.4.4 Lack of Behavioral, Context, and Interaction Analysis

While leading tools focus on identifying malicious content via static indicators, they often fail to assess user behavior or internal and contextual factors. For instance, detection of compromised internal accounts or conversation-based BEC requires behavioral analytics something only a few solutions offer. As attackers increasingly mimic internal users, static filters show clear limitations.

2.5 Conceptual Framework

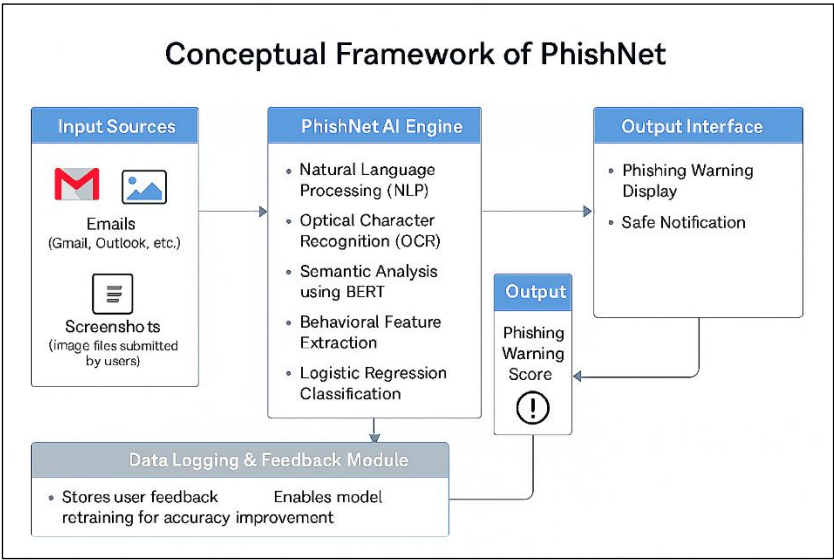


Figure 2.5 Conceptual Framework

PhishNet is an AI-driven phishing detection system that processes emails and user-submitted screenshots to identify potential phishing threats. The system begins with input sources such as

emails from platforms like Gmail or Outlook, and screenshots of suspicious content, which are essential for detecting visually crafted phishing attempts.

These inputs are processed by the PhishNet AI Engine, which includes several intelligent components. Natural Language Processing (NLP) analyzes text in emails for suspicious language or patterns. Optical Character Recognition (OCR) extracts text from screenshots so it can be analyzed. Then, semantic analysis using BERT helps the system understand the context and intent of the message content. Additional behavioral features like sender reputation, link behavior, and metadata are also considered. All of this data is processed through a logistic regression classifier that generates a Phishing Warning Score, indicating the likelihood of a phishing attempt.

The result is then displayed through the Output Interface. If a threat is detected, a Phishing Warning alert is shown to the user. If no threat is found, a Safe Notification confirms the content is legitimate. All outputs are logged, helping improve the system through periodic retraining of the AI models to adapt to evolving phishing techniques

Chapter 3: Development Methodology

3.1 Introduction

This chapter discusses the software development methodology adopted for the PhishNet application and outlines the various aspects of system analysis and design that guide the implementation process. It details the functional and non-functional requirements of the system, the architecture of the phishing detection framework, and the design of key components such as the browser plugin, NLP/OCR modules, and machine learning engine. Additionally, this chapter highlights the development tools and technologies used, as well as the expected system deliverables to be produced upon completion of the application.

3.2 Research Paradigm

This project adopts an experimental research paradigm guided by Object-Oriented Analysis and Design (OOAD), which supports the modular development of PhishNet browser-based phishing detection system. OOAD helps structure the system into clear components such as data preprocessing, AI model training, OCR processing, and user interface, improving maintainability and scalability.

The experimental aspect involves iterative testing of machine learning models, specifically combining BERT for text embeddings and Logistic Regression for classification. Prototypes are developed and evaluated using real phishing datasets, with performance measured through metrics like precision, recall, and F1 score. Tools such as Python in PyCharm, FastAPI, React.js, and GitHub support implementation, testing, and version control throughout the development process.

3.2.1 Data Acquisition

This study uses a combination of text-based and image-based datasets to support robust phishing detection. The Nazario Phishing Corpus provides annotated phishing emails for training models to recognize malicious language, while the Enron Email Dataset offers legitimate emails to balance the classifier and reduce bias. Text from these images is extracted using the Tesseract OCR tool, allowing image inputs to be processed through the same detection pipeline as text, ensuring comprehensive analysis of phishing attempts in multiple formats.

3.2.2 Data Preprocessing

Raw emails undergo preprocessing to extract features suitable for NLP and classification, which is crucial for ensuring model accuracy. Poor preprocessing can significantly impair performance and lead to unreliable predictions. The process begins with cleaning, where HTML tags, duplicates, and noise are removed. This is followed by tokenization and normalization, involving tasks like lowercasing text and removing stop-words. Next, feature extraction is performed, where BERT embeddings are generated to capture semantic meaning, along with additional features such as URL metadata and OCR-extracted text. Data transformation is applied to handle missing values and ensure that all features are properly scaled or encoded for input into the machine learning model (Mastering Data Preprocessing: Techniques and Best Practices, 2023).

3.2.3 Model Training

Once the data is preprocessed, the project trains two complementary models to detect phishing emails. A BERT model is fine-tuned to generate contextual embeddings that capture the semantic meaning of email content. These embeddings, along with additional structural features, are then passed to a Logistic Regression classifier, which outputs the probability of an email being phishing or legitimate. The training process follows standard supervised machine learning practices, incorporating hyperparameter tuning and regularization techniques to improve the model's generalization and performance (codezup, 2024).

3.2.4 Model Validation and Testing

The model is evaluated through both validation and testing to ensure robustness and reliability. During validation, k-fold cross-validation such as 2×5-fold is applied to 70% of the dataset to fine-tune model parameters and assess performance consistency across different data splits (MAxx, 2020). The remaining 30% of the data is used for testing, serving as a hold-out set to simulate real-world performance. Key evaluation metrics include accuracy, precision, recall, and F1 score, along with tracking the rate of false positives (legitimate emails incorrectly flagged as phishing) and false negatives (phishing emails that go undetected). These steps ensure the model performs reliably before deployment.

3.2.5 Experimental Design & Iteration

This paradigm follows a repeatable experimental design. Each sprint conducts controlled experiments testing hypotheses, comparing model architectures, and refining features before deploying the most accurate model in the next sprint(An ‘AI Scientist’ Is Inventing and Running Its Own Experiments | WIRED, 2024).

3.2.6 Tools Summary

PhishNet uses a streamlined set of tools for development and deployment. Python powers the AI models using transformers and scikit-learn, while PyCharm and VS Code support coding and integration. FastAPI handles backend APIs, and Tesseract OCR extracts text from images. Firebase Authentication secures user access, and PostgreSQL manages data storage. Git and GitHub are used for version control and sprint tracking. Together, these tools enable efficient, secure, and scalable development.

3.3 Software Development Methodology

A software development methodology provides a structured approach to planning, building, testing, and delivering software applications. For the development of PhishNet, an AI-driven browser plugin designed to detect and flag phishing emails, this project adopts Scrum-for-One, a tailored adaptation of the Scrum framework for solo developers. While traditional Scrum is designed for collaborative teams, Scrum-for-One preserves its core values transparency, inspection, and adaptation while enabling a single individual to manage all three primary Scrum roles: Product Owner, Scrum Master, and Developer (Lucidchart, 2025).

This methodology is particularly well-suited for cybersecurity and AI-focused projects, which often face dynamic requirements and benefit from iterative development. Research confirms that Scrum-for-One supports self-managed, empirical workflows while maintaining focus on user value and incremental delivery.

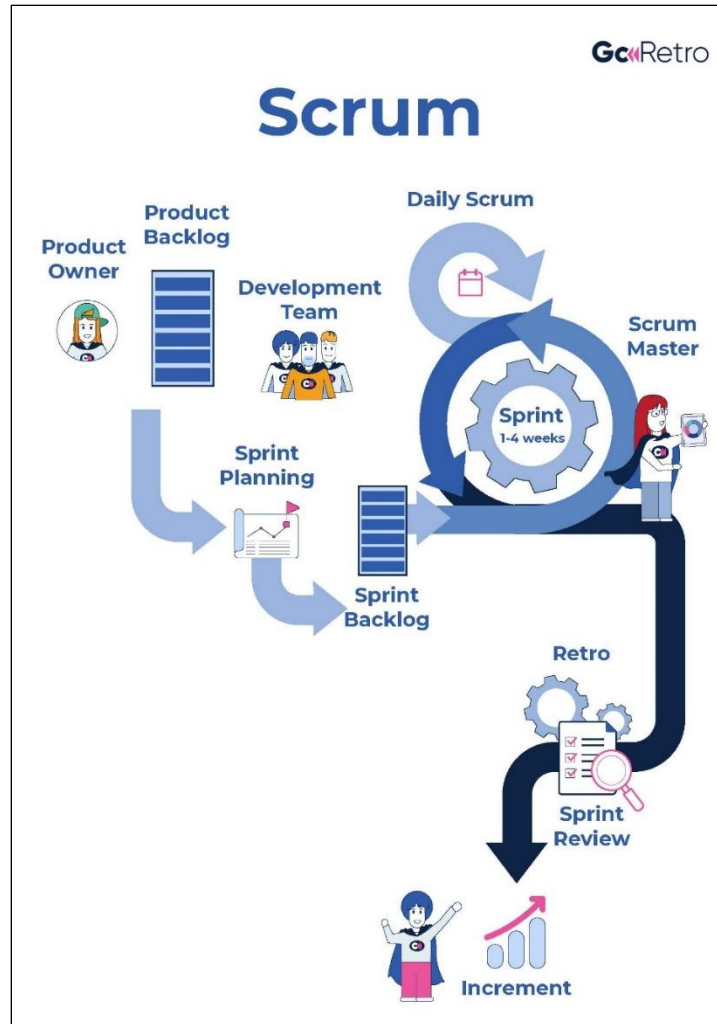


Figure 3.1 Scrum Methodology Diagram(What Is a Scrum Team - GoRetro, 2024)

3.3.1 Product Backlog Creation & Sprint Planning

The methodology begins with the creation of a Product Backlog containing prioritized user stories and tasks such as phishing model development, OCR integration, UI design, and analytics. These backlog items are informed by phishing research, user simulations, and current cybersecurity practices. Example user stories include: “As a user, I want to detect phishing from screenshots” and “As a security analyst, I want to see on demand email phishing risk scores.” Sprint goals are defined and time-boxed to guide execution.

3.3.2 Sprint Design & System Architecture

Each sprint focuses on specific design tasks like wireframes (Figma), backend flows (Draw.io), and plugin UI. Key diagrams include Use Case Diagrams for user interaction, Sequence Diagrams

for process flow, and ERDs for database structure. The system uses a modular architecture, allowing AI components like BERT and Tesseract OCR to function and improve independently.

3.3.3 Incremental Development (Sprint Cycles)

Development progresses through short Sprint Cycles, each delivering a working feature. React.js is used for building the interactive UI, while FastAPI handles backend APIs. Python powers the phishing detection logic using BERT and Logistic Regression. Tesseract OCR extracts text from images, and Firebase Authentication manages secure user login. PostgreSQL stores user and email data. Code is version-controlled with Git and managed on GitHub, with VS Code as the main development environment. Each sprint adds and tests new components like phishing filters, AI models, and browser integration.

3.3.4 Continuous Testing & Sprint Review

At the end of each sprint, a Sprint Review evaluates the deliverables. Testing includes unit tests, integration tests, and usability checks. Emphasis is placed on Test-Driven Development principles where feasible. Each increment is evaluated against sprint goals to ensure real-world readiness and user responsiveness.

3.3.5 Sprint Retrospective & Deployment

Following the review, a Sprint Retrospective is conducted to evaluate what went well, identify areas for improvement, and adjust upcoming sprint plans. Deployment is not part of the scope of this project.

3.4 System Analysis and Design

3.4.1 Use Case Diagram

A use case diagram is essential for visualizing the interactions between different user types (actors) and the system's core functionalities. In the PhishNet system, actors such as regular users, administrators, and the phishing detection engine engage with various modules including email analysis, report generation, profile management, and dashboard viewing. Use case diagrams are particularly helpful in defining system scope by mapping out different scenarios users might encounter. For PhishNet, this diagram clarifies functionalities such as launching the plugin, viewing detection results, updating user profiles, and accessing analytics reports. Its value lies in

simplifying communication between developers and stakeholders by outlining all externally visible behaviors without detailing internal implementation. These insights ensure that developers understand the range of expected system behaviors, enabling more accurate and user-aligned feature development (Use Case Diagram - Unified Modeling Language (UML) - GeeksforGeeks, 2025).

3.4.2 Entity Relationship Diagram (ERD)

The ERD is crucial for designing PhishNet's database architecture, especially as it stores structured data such as user information, phishing simulation results, system reports, and uploaded images. ERDs map out entities (e.g., Users, Reports, Detections, and Images) and define the relationships among them. For instance, a User may be linked to multiple Reports or Detections, while an Image is associated with one detection event. In PhishNet, PostgreSQL will be used to manage these relational tables. ERDs are indispensable when building structured relational database systems, as they improve query optimization, data integrity, and normalization. Implementing an ERD in your system ensures a clear, maintainable, and scalable backend that supports complex queries for analytics and reporting (Cabot, 2023).

3.4.3 Sequence Diagram

A sequence diagram is essential in PhishNet to illustrate how system components and users interact over time, particularly in the context of real-time phishing detection. It captures the flow of operations such as plugin activation, email content or file upload, AI-based classification, and result storage or report generation. These time-ordered interactions between the User, Web Plugin, AI Engine, and Database components help describe the dynamic behavior of the system. Sequence diagrams are vital for modeling communication patterns in modern software systems, especially those that involve asynchronous events, concurrent processes, or interdependent modules. By defining the order of function calls and data exchange, this diagram helps developers implement accurate logic and debug system behaviors more efficiently (Sequence Diagrams, the Only Good Thing UML Brought to Software Development, 2023).

3.4.4 Wireframes / UI Mockups

Wireframes are used to outline the visual structure and flow of the user interface in PhishNet, including key screens such as the dashboard, user profile, phishing detection panel, and report

visualization pages. They provide a low-fidelity representation of how users interact with the system, aiding in early design validation before any development begins. In PhishNet, wireframes ensure that the plugin interface, profile management pages, and analytic dashboards are intuitive, responsive, and user-friendly. Wireframes are indispensable for mobile and web application projects because they align visual structure with usability, thereby enhancing user experience (UX) and enabling rapid prototyping. For PhishNet, wireframes will also be essential when collecting feedback from early testers, helping refine design elements like layout, navigation, and accessibility (Wireframing and Prototyping, 2024).

3.5 System Development Tools

3.5.1 React.js (Frontend Framework)

React.js will be used to build the user interface of PhishNet. Its reusable components and virtual DOM make it ideal for creating a fast, responsive, and interactive user experience for real-time phishing detection dashboards.

3.5.2 FastAPI (Backend Framework)

FastAPI will be employed to build RESTful backend APIs in Python. Its automatic data validation and high performance are essential for handling requests related to email analysis, file uploads, and AI responses.

3.5.3 PyCharm

PyCharm will be used to develop PhishNet's backend and AI components using Python. It supports key libraries like scikit-learn and transformers, and provides efficient tools for coding, debugging, and managing project environments.

3.5.4 Firebase Authentication

Firebase will provide secure user authentication and session management. It simplifies login, registration, and role-based access to system features using both email-password and third-party OAuth providers.

3.5.5 PostgreSQL

PostgreSQL will serve as the primary relational database, storing user data, scanned email logs, classification results, and file metadata. Its reliability, security, and strong query support ensure efficient data handling.

3.5.6 Tesseract OCR

Tesseract OCR will be integrated to extract text from image-based phishing emails (e.g., screenshots). This allows the system to analyze emails that cannot be processed through regular text input.

3.5.7 Git & GitHub

Git will be used for version control, while GitHub will host the repository and track changes, issues, and collaborative contributions during development.

3.5.8 Visual Studio Code

VS Code will be used as the main development environment due to its lightweight interface, support for extensions, and integration with Python, Git, and React.js workflows.

3.5.9 Kaggle

Kaggle will be used to access phishing and email-related datasets for model training and evaluation.

3.6 System Development Techniques

3.6.1 Natural Language Processing (NLP) with BERT

The BERT model will be used to process and classify email content. It enables contextual understanding of messages, helping the system identify subtle phishing cues in text.

3.6.2 Supervised Machine Learning

Supervised learning techniques will be used to train the phishing detection model using labeled datasets. Algorithms like Logistic Regression or BERT will be trained on email content to classify messages as “phishing” or “safe.”

3.6.3 OCR

OCR techniques will be used to analyze screenshots of emails or documents. By converting images into text, the system can apply the same phishing detection logic as it would for regular emails.

3.7 System Deliverables

3.7.1 System Documentation

Comprehensive system documentation will be delivered alongside the PhishNet application to ensure clarity, maintainability, and scalability. This includes the Requirements Specification Document detailing both functional and non-functional system needs, and System Design Documentation describing the architecture, database schema, and key design diagrams such as class and entity relationship diagrams. A User Manual will guide end-users on how to interact with features such as authentication, email uploading, and phishing detection results. Developer Documentation will provide instructions on system setup, API usage, AI model integration, and configuration details. Finally, Testing Reports will document unit, integration, and acceptance tests validating system performance and reliability. Together, these documents provide essential resources for users and future developers.

3.7.2 Web Plugin On-Demand Activation Module

The PhishNet web plugin serves as the central module and includes four integrated components that enhance its functionality and user experience. It contains the Firebase Authentication module, which manages secure user login and identity verification, and the PostgreSQL Image Storage module, responsible for storing screenshots and related scan data captured during phishing detection. Additionally, it includes the User Profile and Dashboard module, which provides users with a personalized interface where they can access phishing detection histories, detailed reports, and security insights. Users can manage their profile information such as names, emails, and profile photos, and configure notification preferences to suit their needs. The plugin also features the Reporting and Analytics module, which compiles and presents comprehensive data on phishing detection activities. It delivers aggregated statistics on detected threats, tracks trends over time, and offers visualizations like graphs and charts to help users and organizations better understand phishing patterns and improve their security posture. All these modules operate under and are accessed through the main web plugin.

3.7.3 AI-Based Phishing Detection Module

At the heart of PhishNet is the AI-Based Phishing Detection Module, which utilizes advanced machine learning and natural language processing techniques to identify phishing content. The module employs models like BERT to analyze email text contextually, detecting deceptive patterns that indicate phishing attempts. For image-based emails or screenshots, Optical Character Recognition (OCR) via Tesseract extracts textual data for analysis. The module also examines embedded links and attachments to detect malicious components. Real-time feedback is provided to users with classification labels and confidence scores, enabling rapid identification of phishing threats. This comprehensive approach allows for effective detection across varied email formats.

Chapter 4: System Analysis and Design

4.1 Introduction

This chapter presents a detailed analysis, design, and implementation of PhishNet the phishing detection and awareness tool designed as a Chrome browser extension. The chapter begins by describing the functional and non-functional requirements that guided development. It then explains the overall system architecture, key modules, and workflows that make the system operational. It provides detailed descriptions of design artifacts such as the use case diagram, ERD, sequence diagram, database schema and user interface wireframes.

4.2 System Requirements

System requirements define the essential capabilities and conditions a system must satisfy to fulfill user needs and operate reliably. According to (Esperón et al., 2024), well-defined requirements form the foundation of effective software design, guiding what the system must achieve and how it should perform under specific constraints. If requirements are unclear or incomplete, the system may malfunction, experience performance issues, or produce inaccurate results. For PhishNet, the system requirements are categorized into functional and non-functional requirements.

4.2.1 Functional Requirements

Functional requirements describe the specific actions, processes, and features that a software system must perform to meet its objectives and support user interaction. According to (Franch et al., 2023), functional requirements define the operations a system carries out in response to user inputs and environmental conditions, forming the basis for validation and testing during implementation. For PhishNet, these requirements were defined to ensure accuracy, security, and smooth operation of the phishing detection system while maintaining intuitive user experience.

The authentication process enables secure user login and registration through Firebase Authentication. When a user registers, passwords are hashed and encrypted to protect account credentials. After authentication, users can access their past phishing scans, manage stored data, and maintain persistent sessions across logins. Login in enhances personalization and data consistency.

The phishing detection process represents the system's core functionality. Users initiate a scan by opening an email on a web platform and selecting the Scan option through the PhishNet interface.

The system then captures the visible email content and runs it through a multi-stage detection pipeline that includes OCR, text preprocessing, and contextual embedding using the BERT model. The model analyzes the input and returns a verdict Safe or Phishing along with a confidence percentage, which is displayed to the user through the interface.

The OCR processing process extends phishing detection to image-based threats such as fake login screenshots. It uses Tesseract OCR 5.3 to extract text embedded in uploaded images. Before analysis, the system applies preprocessing steps such as grayscale conversion, noise reduction, and text sharpening to improve extraction accuracy. The extracted text is then analyzed using the same BERT-based detection pipeline as textual inputs.

The result logging and history process securely stores each scan result together with its timestamp, confidence score, and extracted features. Users can access their previous results through a history section, which promotes transparency and allows them to review or delete stored records. This process ensures traceability of detections and supports performance monitoring.

The user interface process focuses on clarity, accessibility, and ease of interaction. It provides intuitive controls such as a Scan button, visual indicators, and color-coded results green for safe content and red for phishing. Feedback messages and icons help users understand the outcomes, while options to clear or export scan results give them greater control over their data.

The browser integration process ensures that PhishNet operates seamlessly through a lightweight plugin interface. The system is optimized for fast performance and minimal resource consumption so that it does not interfere with normal browsing. The plugin appears as an icon in the browser toolbar, giving users quick access to the scanning feature directly from an email page.

4.2.2 Non-Functional Requirements

Non-functional requirements define the quality standards and operational conditions that ensure PhishNet runs effectively and efficiently. According to (Keong & Embi, 2022) these requirements describe how well a system performs its functions in terms of security, performance, usability, reliability, and data privacy. For PhishNet, they ensure that the system remains secure, stable, and user-friendly throughout its operation.

System security is crucial for protecting user data and maintaining trust. PhishNet uses Firebase Authentication for secure login and encrypts all stored passwords to prevent unauthorized access.

Access to backend resources is controlled through API keys and role-based permissions, ensuring that only authorized users can access specific system functions. These measures help safeguard user accounts and protect the system from malicious activities.

Performance focuses on ensuring that PhishNet responds efficiently under different workloads. The system has been optimized to deliver phishing detection results quickly and without unnecessary lag. This is achieved by streamlining database queries and improving the model's processing speed, allowing users to receive results promptly and maintain a smooth browsing experience.

Usability ensures that the application is simple and clear for both technical and non-technical users. The interface allows scans to be initiated with one click or by dragging and dropping an email or screenshot. The feedback uses intuitive color codes green for safe, yellow for suspicious, and red for phishing to communicate results clearly and help users act quickly.

Reliability and availability ensure that PhishNet remains stable and accessible at all times. Once installed, it is available directly from the browser toolbar. A background process checks for internet connectivity and displays friendly error messages when a network failure occurs, rather than allowing the system to crash. This keeps the system dependable even under poor network conditions.

Data privacy and confidentiality ensure that sensitive user information is always protected. PhishNet never uploads full email content; instead, only extracted features necessary for classification are sent for processing. The system also anonymizes personal identifiers such as usernames and email addresses in the results to protect user identity and maintain confidentiality.

4.3 System Narrative

The PhishNet system provides a phishing detection solution that operates through a browser interface. Its primary purpose is to analyze email content and detect potential phishing attempts by evaluating both textual and image-based inputs. The system follows a user-centered workflow that begins when a user opens an email on a web platform such as Gmail or Outlook and activates the Scan option through the PhishNet plugin. Once triggered, the plugin automatically extracts the visible content of the open email including the subject line, body text, and embedded links and

sends it securely to the backend for analysis. Alternatively, users can upload a screenshot of a suspicious email in the dashboard to be analysed.

After receiving the input, the backend applies a multi-stage analysis pipeline. If the input is an image, the system uses Tesseract OCR to extract any embedded text. The extracted text is passed into the BERT model, which generates contextual embeddings representing the semantic meaning of the content. The model analyzes these embeddings and classifies the input as either phishing or legitimate based on learned patterns.

Once classification is complete, the system returns the results to the user interface, displaying a clear verdict and a confidence score. Color-coded visual feedback such as green for safe content and red for phishing helps the user quickly interpret the results. Each scan result, along with metadata such as timestamps and classification outcomes, is securely stored in the backend database. Users can later review their detection history or export summaries for reference. Administrators can also monitor overall system performance and usage statistics through the analytics dashboard.

The PhishNet system narrative outlines an end-to-end process that starts with a user scanning an email directly from a web page and concludes with a clear, actionable phishing detection result. This streamlined workflow ensures security, efficiency, and accuracy while maintaining an intuitive and user-friendly experience.

4.4 System Design

The system design phase converts the analyzed requirements into visual and logical representations that define how PhishNet's components interact and function. It uses key design diagrams: Use Case, ERD, Sequence, and Database Schema to illustrate user interactions, data relationships, process flow, and database structure. Together, these diagrams provide a clear blueprint for the system's architecture and ensure all components work cohesively toward the project's objectives.

4.4.1 Use Case Diagram

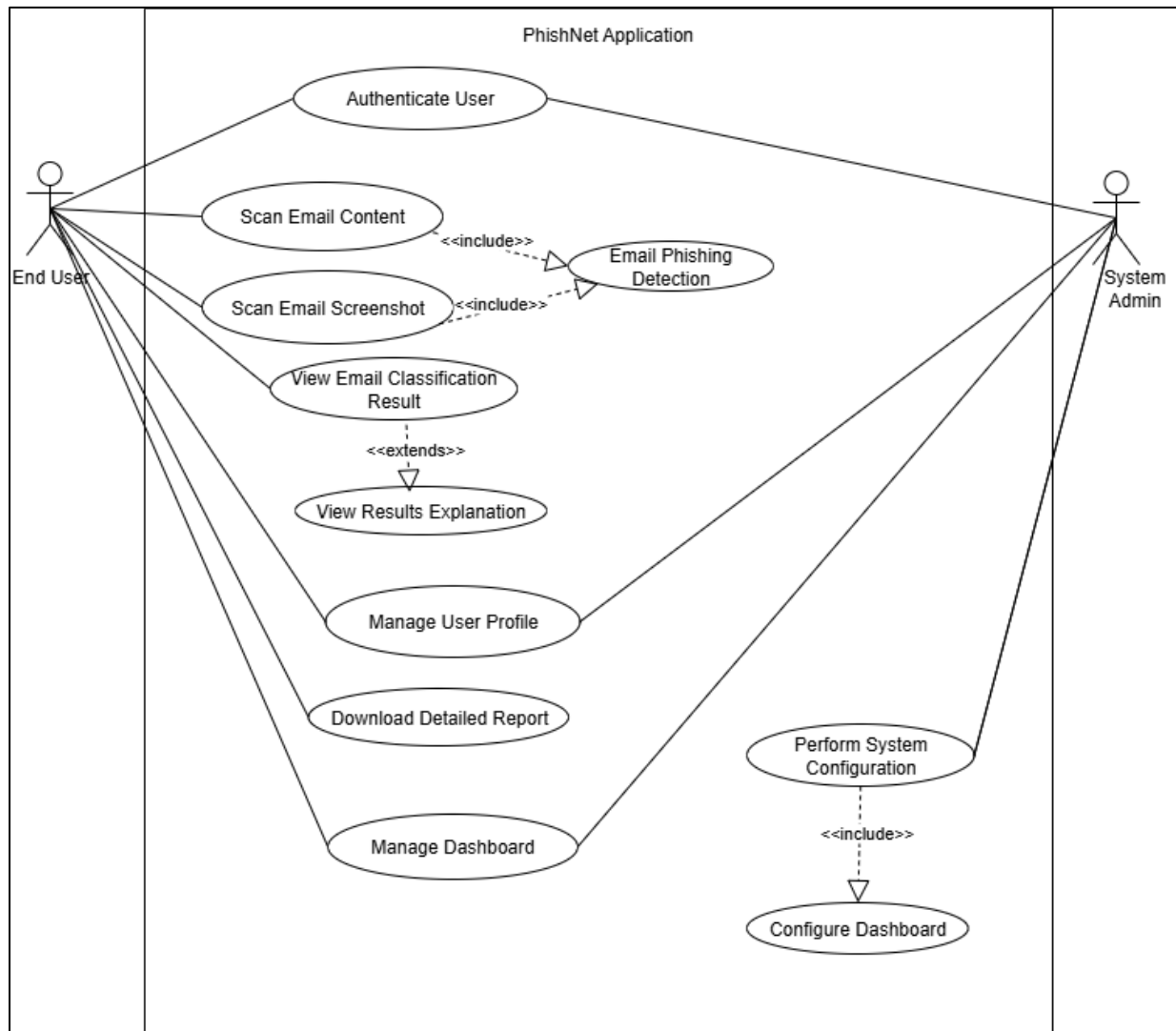


Figure 4.1 Use Case Diagram

The use case diagram for PhishNet illustrates the interactions between two primary actors the End User and the System Admin and the system's core functionalities. The End User initiates interaction by authenticating into the system, after which they can perform phishing detection through two primary methods: scanning email content directly or scanning email screenshots. Both scanning methods include the Email Phishing Detection use case, indicating that the AI-driven detection process is automatically triggered whenever a scan is initiated, making it a required component of the scanning workflow. Once detection is complete, users can view the email classification result, which displays whether the content is phishing or safe along with a confidence

score. This use case is optionally extended by the View Result Explanation feature, allowing users to access detailed insights into why the system flagged the content as phishing or safe, though this step is not mandatory. Users can also manage their profiles, download phishing reports for documentation purposes, save result logs for future reference, and access the Manage Dashboard feature to view analytics and scan history.

The Admin actor shares several use cases with the End User, including authentication, profile management, and dashboard access, but has exclusive access to the Perform System Configuration use case, which includes the Configure Dashboard function. This include relationship indicates that configuring the dashboard is a necessary part of system configuration, ensuring that admins can customize system settings and dashboard layouts to meet organizational needs. The diagram effectively demonstrates how PhishNet provides comprehensive phishing detection capabilities for everyday users while granting administrators the additional control needed to maintain and optimize system performance.

4.4.2 Entity Relationship Diagram (ERD)

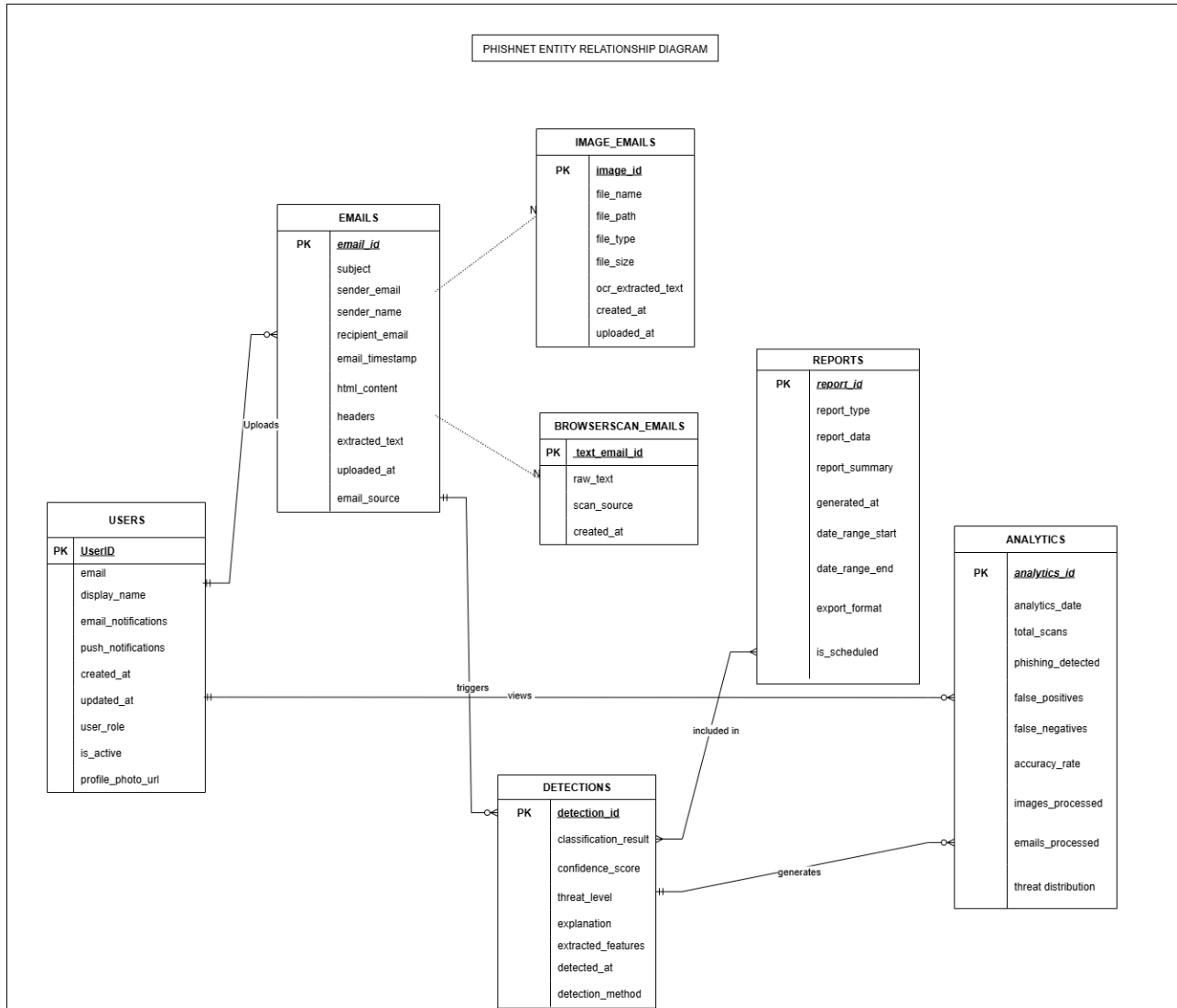


Figure 4.2 Entity Relationship Diagram

The PhishNet ERD illustrates how entities interact within the phishing detection system in a real-world context. At the center of the diagram is the Users entity, representing individuals who interact with the system by uploading suspicious email content for analysis. Users can upload emails for scanning, establishing a one-to-many relationship where a single user can submit multiple emails over time.

Each email scan falls into one of two categories: BrowserScan_Emails, which handles text content directly captured from the browser plugin, or Image_Emails, which processes screenshots of

suspicious emails that require OCR text extraction. These two specialized email types act as weak entities dependent on the main emails entity, clearly distinguishing between the two different input methods. Both text-based and image-based emails trigger the Detections entity, meaning each uploaded item undergoes a single BERT-powered analysis that produces a classification result, confidence score, threat level, and detailed explanation.

The detection results feed into the Analytics entity through a many-to-one relationship, where multiple individual detections contribute to aggregated metrics such as total scans, phishing detected, accuracy rates, and threat distribution patterns. Users can generate Reports through a one-to-many relationship that compiles detection data into downloadable summaries for specific time periods. These reports can include multiple detections while each detection may appear in multiple reports, creating a many-to-many relationship. Users can view the analytics dashboard to monitor system performance and track phishing trends over time. This reflects the complete workflow from user-initiated scanning through detection to comprehensive reporting and analytics, capturing how PhishNet processes and manages phishing threats in a structured, traceable manner.

4.4.3 Sequence Diagram

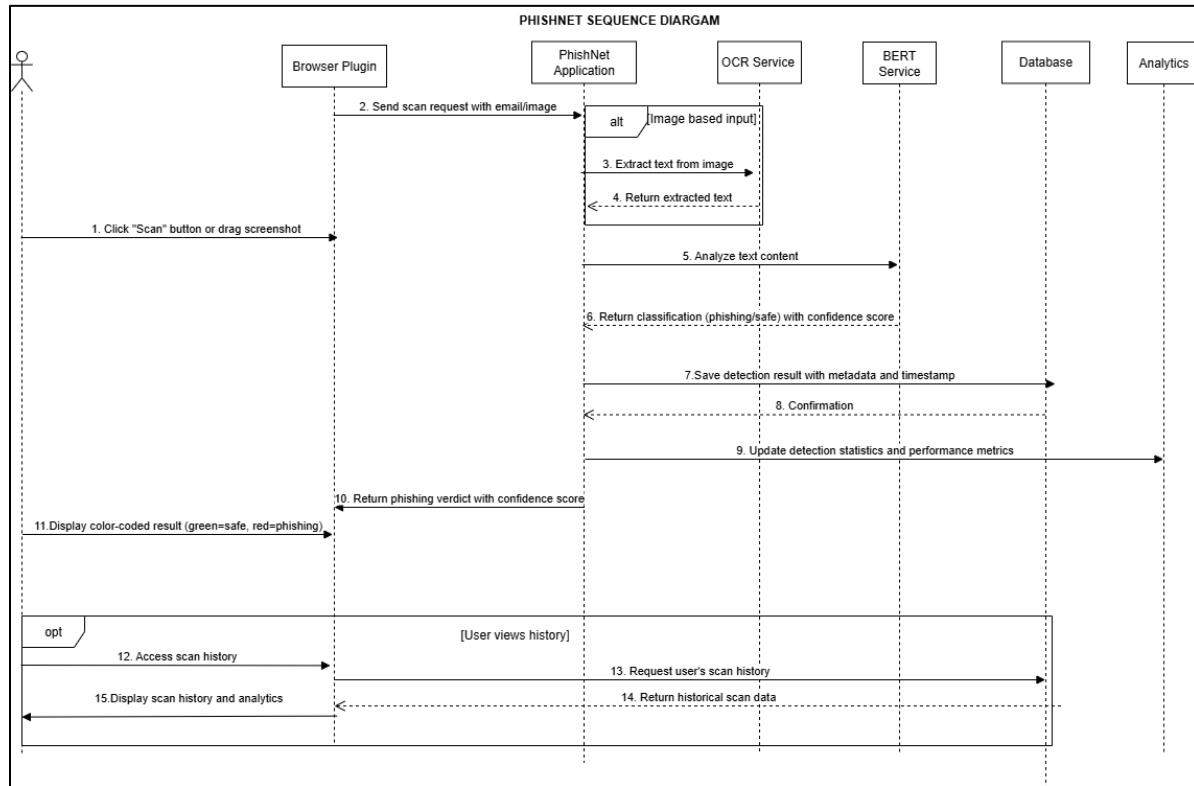


Figure 4.3 Sequence Diagram

The sequence diagram for PhishNet illustrates the dynamic flow of interactions between the user, the browser plugin, the PhishNet application, external services (OCR and BERT), and the database. The process begins when the user initiates a scan by clicking the Scan button or dragging a screenshot into the plugin interface. The browser plugin captures the input and transmits it securely to the PhishNet Application backend. An alternative (alt) frame represents conditional processing: if the input is an image, the system routes it through the OCR Service for text extraction using preprocessing techniques like grayscale conversion and noise reduction; if the input is already text-based, it bypasses OCR entirely. Once text is available, it is sent to the BERT Service, which performs unified detection by generating contextual embeddings and immediately classifying the content, returning both a classification label (phishing or safe) and confidence score in a single streamlined interaction.

The PhishNet Application then logs the scan details including extracted text, probability scores, and timestamps into the PostgreSQL Database for traceability, while simultaneously updating the

Analytics module with detection metrics. The result is returned to the browser plugin and displayed to the user through a color-coded interface: green for safe emails and red for phishing alerts. An optional (opt) frame shows that users may choose to access their scan history at any time, where the plugin retrieves and displays past scan records from the database, providing insights into previous detections. This workflow demonstrates how PhishNet automates phishing detection through an intelligent, adaptable, and secure process from user initiation to classification and persistent logging.

4.4.4 Database Schema

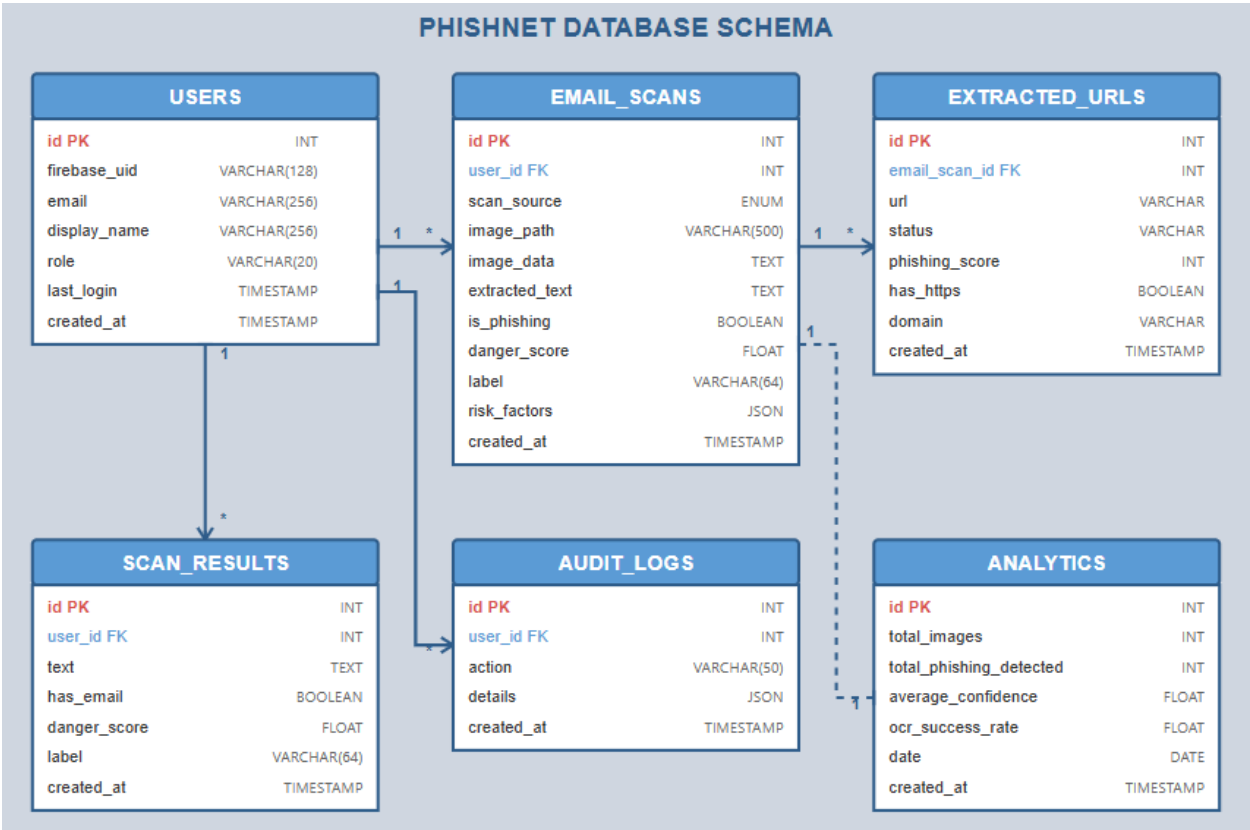


Figure 4.4 Database Schema

The PhishNet database is organized around six main tables that work together to handle user management, phishing detection, and system monitoring. At the center is the Users table, which stores basic information like email addresses, display names, and Firebase authentication IDs. This

table connects Firebase Authentication to handle secure logins and keeps track of when users last accessed the system and what role they have, like admin or regular user.

When a user scans an email or image for phishing, that information goes into the Email_Scans table. This table is comprehensive, it records where the scan came from whether browser scan or OCR image scan , stores the actual image data, saves the text extracted by the OCR system, and most importantly, keeps the results from the BERT model including whether it's phishing or legitimate, the confidence score, and specific risk factors that were detected. The Extracted_URLs table works alongside this to store every URL found in the scanned content, giving each one its own threat score and checking things like whether it uses HTTPS or comes from a suspicious domain.

The Scan_Results table provides a cleaner summary of each detection, storing the final verdict and danger score for easy retrieval when users want to view their scan history. The Audit_Logs table keeps a detailed record of every action taken in the system: logins, scans, setting changes which is crucial for security monitoring and troubleshooting. The Analytics table aggregates everything into useful statistics like total scans performed, phishing attempts detected, and average confidence scores, helping track how well the system is performing over time.

Everything connects logically through foreign keys users own their scans, scans contain URLs and produce results, and all actions get logged. This structure makes it easy to trace any scan back to its user, analyze patterns in phishing attempts, and maintain a complete audit trail while keeping queries fast and efficient.

4.5 System Wireframes

The PhishNet wireframes are high-fidelity design blueprints that represent how the system is intended to look and function once developed. They outline the planned layout and interface flow across four main views: the browser plugin popup, the web dashboard, the detailed scan results page, and the user profile and settings page. These wireframes serve as detailed visual guides for development, emphasizing simplicity, clarity, and responsiveness to ensure that both technical and non-technical users will be able to interact with the phishing detection system easily and intuitively.

The Browser Plugin Popup Interface provides the most direct interaction point for users. It contains an upload area where users can drag-and-drop or browse for email screenshots, alongside a one-click button to scan directly from the browser. The interface is deliberately minimal, with bold call-to-action buttons and a results panel that displays detection outcomes using color-coded alerts: green for safe emails, amber for warnings, and red for phishing threats. The confidence score is visualized with a progress bar, making it easy for users to quickly gauge system certainty. Supplementary buttons such as View History and Settings enhance accessibility without overwhelming the main workflow. This is seen in Figure 4.5.

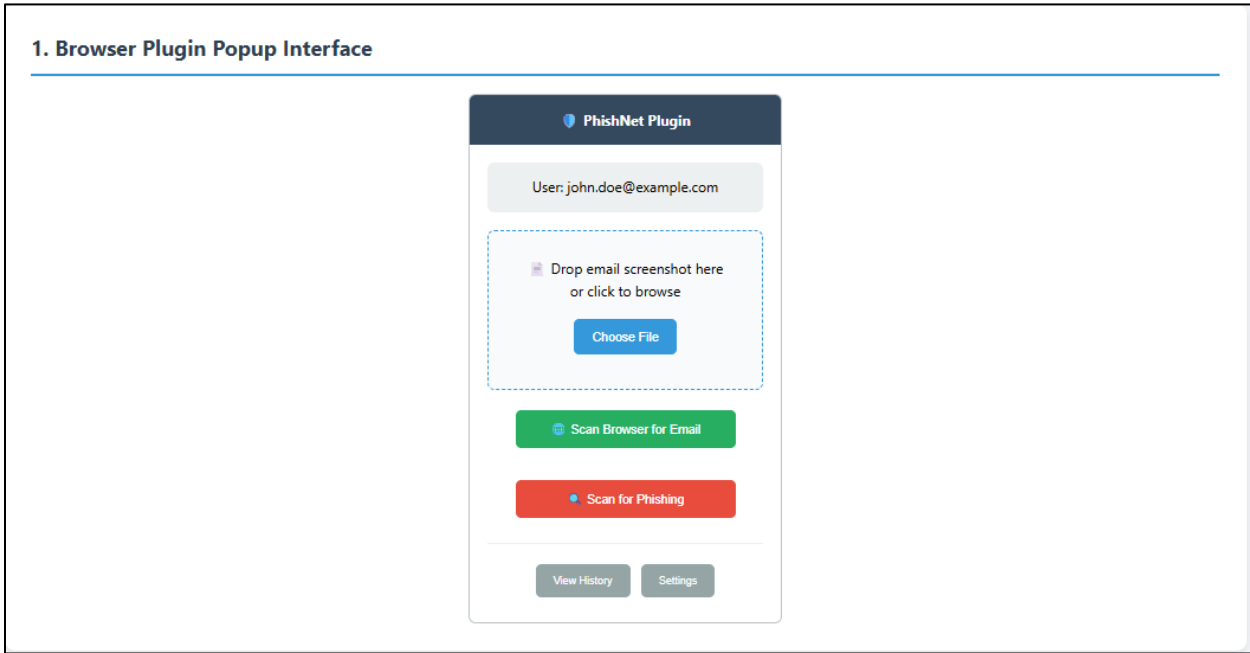


Figure 4.5 Browser Plugin Popup Interface

The Web Dashboard Interface offers a broader view of the system’s performance and user activity. A sidebar navigation menu provides structured access to different sections such as Overview, Scan History, Analytics, Reports, and Profile. The main dashboard area displays summary statistics using visual cards (e.g., total scans, threats detected, accuracy rate, and images processed). Charts for Threat Trends and Threat Distribution allow users to observe phishing patterns over time, while the Recent Scans panel provides contextual insights into the latest results. This dashboard supports both individual learning and organizational-level monitoring. This is seen in Figure 4.6.

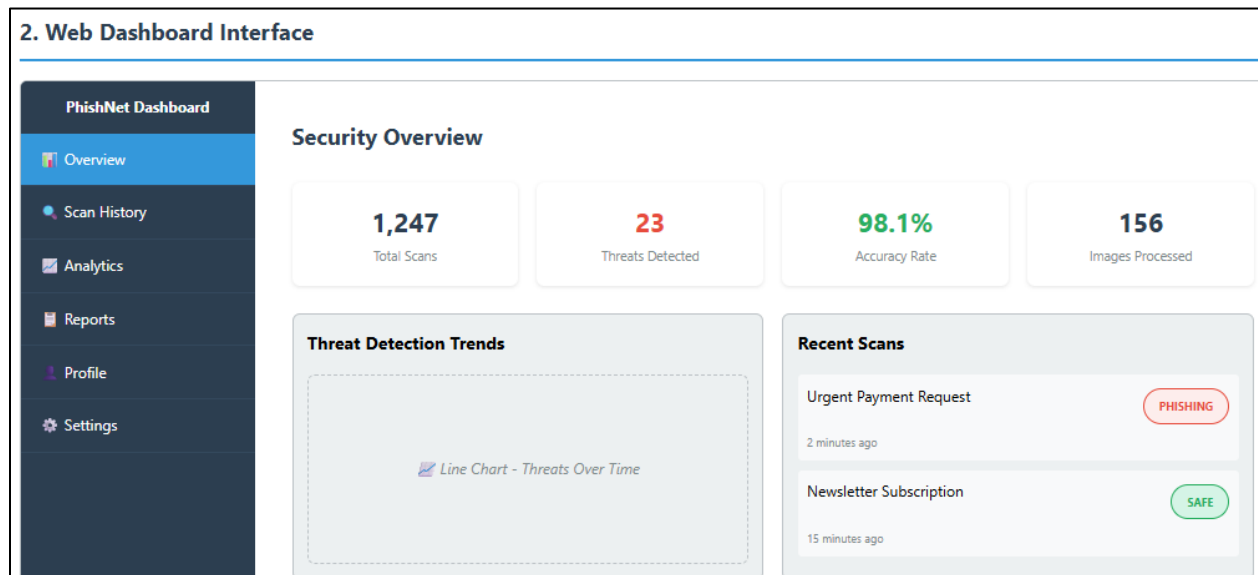


Figure 4.6 Web Dashboard Interface

The Detailed Scan Results Interface focuses on transparency and actionable feedback. A scan summary displays key metadata such as sender, subject line, and timestamp, accompanied by a high-visibility phishing verdict and confidence score. Threats detected (e.g., domain spoofing, urgency language, suspicious links) are highlighted in status indicators, while an AI analysis panel provides deeper insight into BERT sentiment outputs, OCR confidence, and overall classification. At the bottom, actionable buttons allow users to delete suspicious emails, report phishing, save results, or provide feedback. This design balances technical detail with clear guidance, helping users not only detect but also respond effectively to threats. This is seen in Figure 4.7

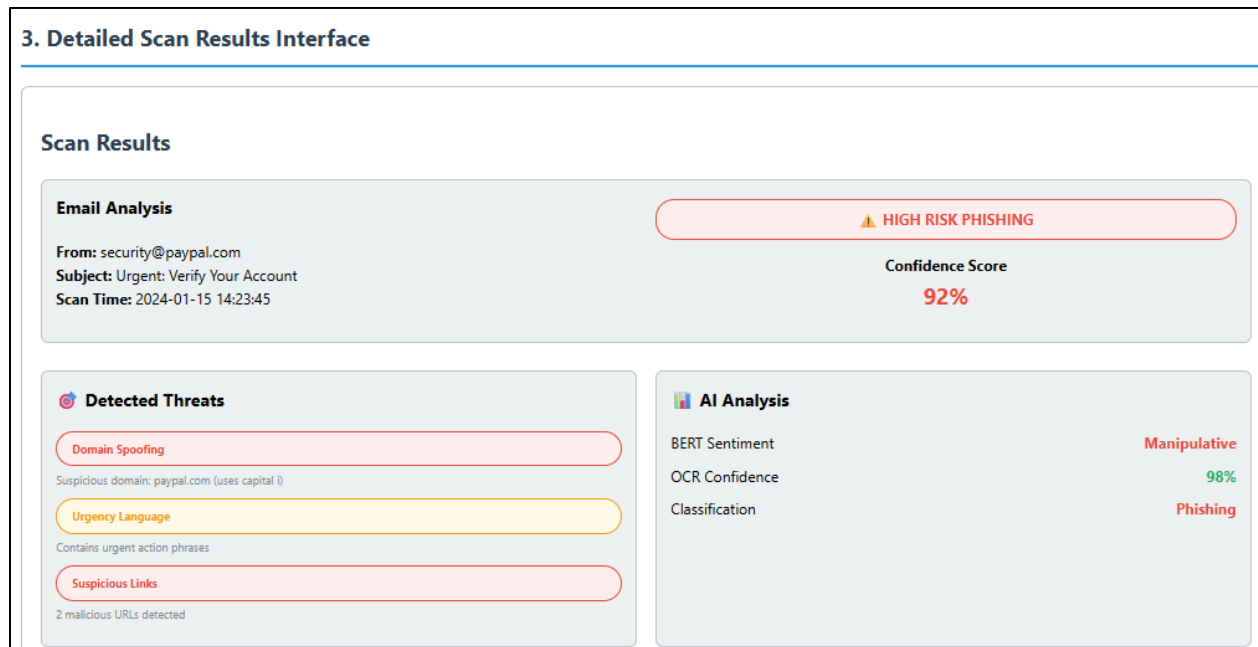


Figure 4.7 Detailed Scan Results Interface

The User Profile and Settings Interface supports personalization and customization. The profile card allows users to edit personal details and update profile photos, while the notification settings panel provides granular control over alerts, such as email notifications, push alerts, weekly reports, and accuracy feedback. Detection sensitivity can also be tuned here, enabling advanced users to adjust the system’s strictness according to their risk appetite. Together, these features enhance both usability and trust, ensuring that users feel in control of their data and interactions with the system. This is seen in Figure 4.8.

4. User Profile & Settings

User Profile



Change Photo

Save Changes

Notification Settings

☒ Email notifications for threats

☒ Browser push notifications

☐ Weekly security reports

☒ Detection accuracy feedback

Detection Sensitivity

Figure 4.8 User Profile and Settings Interface

4.6 System Architecture

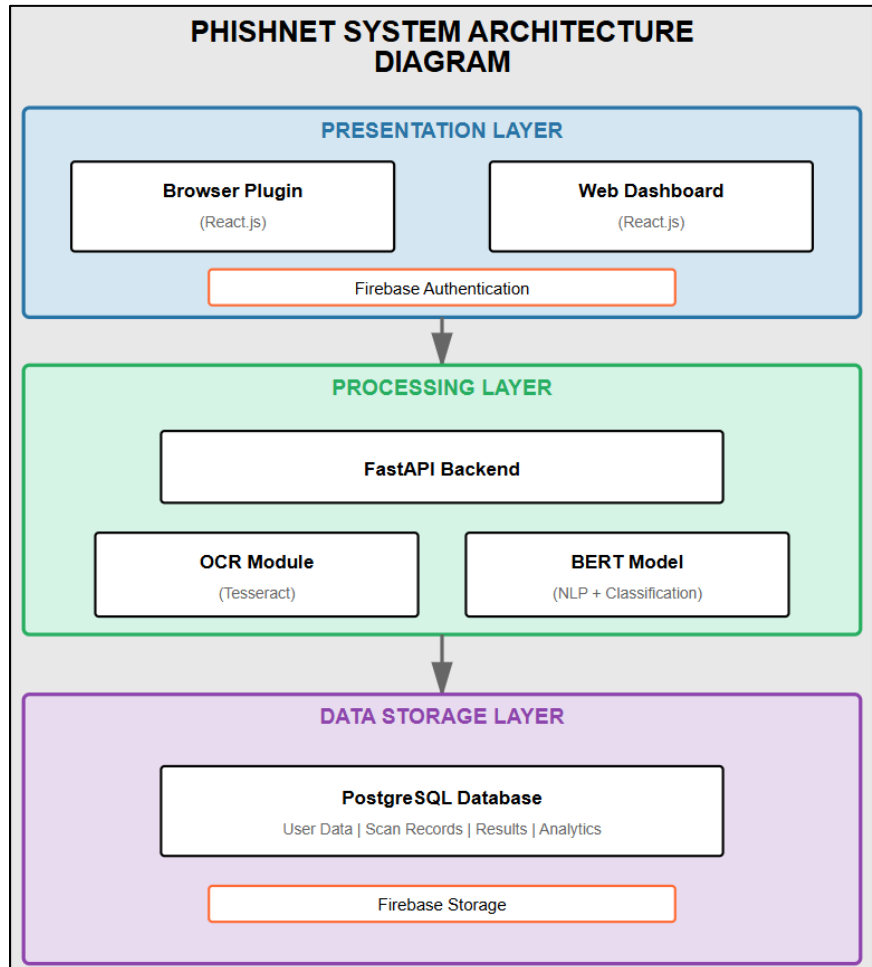


Figure 4.9 System Architecture Diagram

PhishNet is built on a modified three-tier architecture that separates concerns across presentation, processing, and data storage layers. The Presentation Layer consists of two client-side components: a Chrome browser plugin and a web-based dashboard, both developed using React.js to provide an intuitive and responsive user interface. Firebase Authentication secures both interfaces by managing user login, registration, and session control, and also stores user authentication data, which is why firebase storage is included in the Data Storage Layer. The Processing Layer houses the core intelligence of the system, implemented through a FastAPI backend server that manages all business logic and API endpoints. This layer integrates two critical modules: the OCR Module, powered by Tesseract, which extracts text from image-based phishing attempts such as screenshots, and the BERT Model, which performs natural language processing

and classification to detect phishing content based on semantic analysis. The Data Storage Layer utilizes PostgreSQL as a relational database to persistently store user profiles, scan history, detection results, and analytics data. Communication between the presentation and processing layers occurs via secure HTTPS REST APIs run by FastAPI, while the processing layer interacts with the database through SQL queries. This layered architecture ensures modularity, scalability, and clear separation of responsibilities, allowing each component to be developed, tested, and maintained independently while working together to deliver, on-demand phishing detection directly within the user's browser.

Chapter 5: System Implementation and Testing

5.1 Introduction

This chapter presents the implementation and testing of PhishNet, a phishing detection system. It outlines the environment in which the system was developed and tested, including both hardware and software configurations. The chapter also describes the dataset used for model training and evaluation, detailing how data was prepared and validated. It discusses the integration of the trained model into the web-based plugin interface and explains the testing processes carried out to verify functionality, accuracy, and user experience. The main goal of this phase was to ensure that PhishNet performs efficiently, delivers accurate phishing classifications, and maintains system stability across different operating environments.

5.2 Implementation Environment

5.2.1 Hardware Specifications

Table 5.1 Hardware Specifications

Component	Test Case Requirements	Recommended Requirements
Processor	Intel® Core™ Ultra 7 256V (2.20 GHz, x64-based)	Intel® Core™ i7 / Ryzen 7 (3.0 GHz or higher), 8+ cores
RAM	16 GB	16–32 GB (32 GB ideal for intensive workloads)
Storage	1 TB SSD	512 GB – 1 TB NVMe SSD
Graphics	Integrated Intel® Arc GPU	Dedicated GPU optional (NVIDIA GTX 1650 or better)
Network	802.11ac Wi-Fi	802.11ax (Wi-Fi 6)

Display	14.0” Full HD	15.6” Full HD or higher (1080p/1440p)
Peripheral Devices	Mouse, Keyboard	Mouse, Keyboard, External Monitor
System Type	64-bit Operating System, x64 Architecture	Same requirement; Windows 11 or Ubuntu 22.04 recommended

Table 5.1 presents the hardware specifications of the test machine used during system development and evaluation, along with recommended hardware requirements. These recommendations are based on the computational needs of OCR processing, machine learning inference, browser-based scanning, backend services, and administrative analytics within the PhishNet system.

5.2.2 Software Specifications

Table 5.2 Software Specifications

Component	Specification / Version Used
Operating System	Windows 11 (64-bit)
Programming Language	Python 3.10, JavaScript (ES6)
Development Frameworks	FastAPI (Backend), React.js (Frontend)
Database Server	PostgreSQL 15
OCR Engine	Tesseract OCR 5.3
Authentication	Firebase Authentication
IDEs	PyCharm, Visual Studio Code
Version Control	Git and GitHub
Browser Testing Environment	Google Chrome 142.0 (Developer Mode)
AI Model Used	DistilBERT (Hugging Face Transformers)

AI Libraries	Transformers, Scikit-learn, Pandas, NumPy
Additional Tools	BeautifulSoup4, Regex, Matplotlib, JSON, OS libraries

Table 5.2 lists the software components used during the development and testing of the PhishNet system. At the time of development, it is recommended that users install the most recent and stable versions of these software tools to ensure optimal performance, security, and compatibility.

5.3 Dataset Description and Preparation

PhishNet’s model was trained using three trusted text-based datasets obtained from Kaggle, each contributing to a diverse and realistic representation of phishing and legitimate communication. These datasets were selected to improve the system’s ability to identify malicious patterns, reduce false positives, and enhance model generalization. By combining phishing emails, legitimate corporate communication, and mixed spam/ham messages, PhishNet’s dataset mirrors the variety of emails that users encounter in real environments.

5.3.1 The Nazario Phishing Corpus

The Nazario dataset contains real phishing emails collected from active malicious campaigns. It includes the fields sender, receiver, date, subject, body, and URLs which is displayed in Figure 5.1, all of which play a meaningful role in capturing the structure of typical phishing behaviors.

The sender field frequently contains spoofed names or domains crafted to look legitimate, making it useful for understanding impersonation tactics. The receiver shows the targeted address, which can reveal mass-targeting patterns. The date of the email helps track how phishing campaigns evolve across time. The subject often contains urgent, threatening, or manipulative language intended to pressure the reader into acting quickly. The body contains the main content of the phishing message this is the most important part for training, as it carries the text patterns, tone, and wording of malicious intent. The URLs field reveal malicious or deceptive links embedded inside phishing emails. Since URLs are one of the strongest phishing indicators, this field is extremely valuable for detection.

Nazario.csv (7.81 MB) Download Icon Fullscreen Icon More Icon

Detail **Compact** Column 7 of 7 columns

About this file

Contains Nazario Phishing Email Dataset. The features include sender, receiver, date, subject, body, urls.

✉ sender Sender Email	✉ receiver Receiver Email	📅 date Date Time	📄 subject Subject
--------------------------	------------------------------	---------------------	----------------------

Figure 5.1 Nazario Phishing Corpus Dataset

5.3.2 The Enron Email Dataset

The Enron dataset provides legitimate internal corporate communication. It includes the fields: subject and body, representing authentic business conversations, this is displayed in Figure 5.2. The subject lines typically contain neutral, professional descriptions without emotional manipulation. The body messages provide natural work-related communication patterns, including formal tone, collaborative phrases, and standard information-sharing formats. By learning from these, the model develops an understanding of what safe, non-malicious email communication looks like, improving its ability to avoid false positives.

Enron.csv (45.56 MB) Download Icon Fullscreen Icon More Icon

Detail **Compact** Column 3 of 3 columns

About this file

Contains the Enron dataset. The features include subject, body.

Figure 5.2 Enron Email Dataset

5.3.3 The SpamAssassin Public Corpus

The SpamAssassin dataset contains a mixture of spam and legitimate emails, using fields sender, receiver, date, subject, body, and URLs, this is displayed in Figure 5.3. The sender field often

contains auto-generated or promotional email patterns. The receiver field reflects wide distribution, common in mass spam campaigns. The date helps analyze timing patterns in spam bursts. The subject lines frequently contain exaggerated claims, promotions, or attention-grabbing hooks. The body exposes the model to a wide range of spam content, some harmless, some suspicious, which helps it learn the difference between general spam and targeted phishing attacks. The URLs field contains embedded links in spam, which may look like phishing links but are usually designed for advertising. This variety helps the model learn subtle differences between spam and phishing.

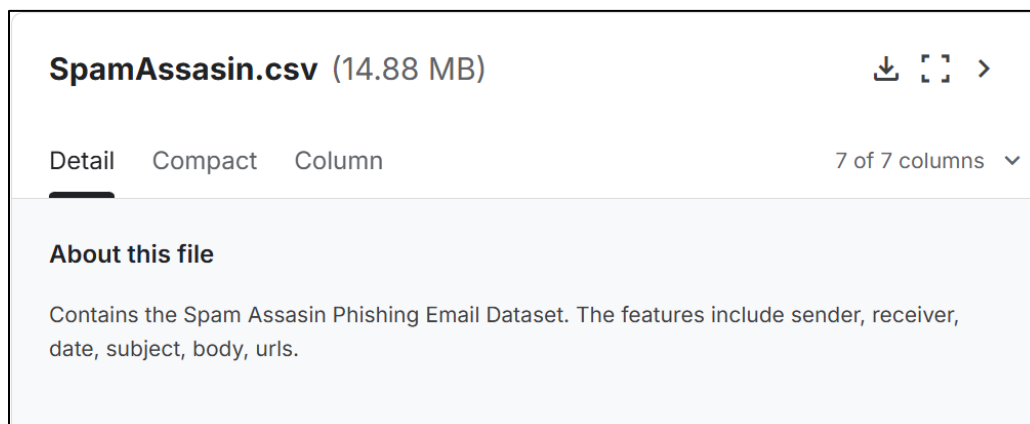


Figure 5.3 SpamAssassin Dataset

After cleaning and formatting, the three datasets were merged into a single, unified dataset. Duplicate records, empty entries, and non-English emails were removed to ensure data quality and consistency. The final dataset contained columns such as subject, body, and label, representing email content and its classification as phishing or legitimate. The screenshot in Figure 5.4 from VS Code shows the final dataset structure and number of rows.

```
Loading original datasets...
Loading Enron dataset...
  Enron loaded: 29767 emails
Loading Nazario dataset...
  Nazario loaded: 1565 emails
Loading SpamAssassin dataset...
  SpamAssassin loaded: 5808 emails
  Combined: 37,140 emails
  SpamAssassin loaded: 5808 emails
  Combined: 37,140 emails
```

Figure 5.4 Datasets Structure

5.3.4 Dataset Cleaning and Preprocessing

The dataset preparation process for PhishNet was automated using a custom Python script named `prepare_dataset.py`. This script was designed to load, clean, merge, and balance the three datasets Enron, Nazario, and SpamAssassin into a single, standardized dataset suitable for training the phishing detection model. Each step of the cleaning pipeline was implemented carefully to ensure the resulting data was accurate, consistent, and ready for machine learning.

The process began by loading the raw datasets using dedicated loader functions. Each dataset was read into a Pandas Data Frame, missing text fields were removed, and column names were standardized. A new column called `text` was created by combining the subject, body, and URL fields of each email. This ensured that every entry represented a complete message. The code functions used to achieve this are shown in Figure 5.5.

```

def load_enron():
    print("Loading Enron dataset...")
    df = pd.read_csv(ENRON_PATH)
    df = df.dropna(subset=["body"])
    df["subject"] = df["subject"].astype(str)
    df["body"] = df["body"].astype(str)
    df["label"] = df["label"].astype(int)
    df["text"] = df["subject"].fillna("") + " " + df["body"].fillna("")
    print(f"Enron loaded: {len(df)} emails")
    return df[["text", "label"]]

def load_nazario():
    print("Loading Nazario dataset...")
    df = pd.read_csv(NAZARIO_PATH)
    df = df.dropna(subset=["body"])
    for col in ["subject", "body", "urls"]:
        if col in df.columns:
            df[col] = df[col].astype(str)
        else:
            df[col] = ""
    df["label"] = df["label"].fillna(1).astype(int)
    df["text"] = df["subject"].fillna("") + " " + df["body"].fillna("") + " " + df["urls"].fillna("")
    print(f"Nazario loaded: {len(df)} emails")
    return df[["text", "label"]]

def load_spamassassin():
    print("Loading SpamAssassin dataset...")
    df = pd.read_csv(SPAMASS_PATH)
    df = df.dropna(subset=["body"])
    for col in ["subject", "body", "urls"]:
        if col in df.columns:
            df[col] = df[col].astype(str)
        else:
            df[col] = ""
    df["label"] = df["label"].fillna(1).astype(int)
    df["text"] = df["subject"].fillna("") + " " + df["body"].fillna("") + " " + df["urls"].fillna("")
    print(f"SpamAssassin loaded: {len(df)} emails")
    return df[["text", "label"]]

```

Figure 5.5 Dataset loading functions for Enron, Nazario, and SpamAssassin.

Once the datasets were loaded, they were merged into a single DataFrame. The next step involved text normalization using regular expressions and BeautifulSoup. This stage removed HTML tags, replaced sensitive items such as URLs, email addresses, and numbers with placeholders (<URL>, <EMAIL>, <NUM>), and converted all text to lowercase. It also trimmed overly long messages to a consistent length of 3,500 characters to maintain uniform input size. The code function used to achieve this is shown in Figure 5.6.

```

def normalize_text(text):
    if not isinstance(text, str):
        text = str(text)
    text = BeautifulSoup(text, "html.parser").get_text(separator=" ")
    text = URL_RE.sub(" <URL> ", text)
    text = EMAIL_RE.sub(" <EMAIL> ", text)
    text = NUM_RE.sub(" <NUM> ", text)
    text = MULTI_WS_RE.sub(" ", text).strip().lower()
    if len(text) > MAX_TEXT_LEN:
        text = text[:MAX_TEXT_LEN]
    return text

```

Figure 5.6 Text Normalization Function for Cleaning and Standardizing Emails.

After cleaning, the script applied language detection to retain only English-language emails. The langdetect library was used to identify non-English samples, which were removed to improve the linguistic consistency of the dataset. The code used to achieve this is shown in Figure 5.7.

```
def is_english(text):
    try:
        return detect(text[:4000]) == "en"
    except Exception:
        return False
```

Figure 5.7 Language Filtering Process.

To make the dataset more representative of real-world conditions, the script generated over 200 synthetic legitimate emails using a custom function named generate_realistic_emails(). These examples covered everyday topics such as e-commerce, subscriptions, shipping, and corporate communication, making the dataset more balanced and realistic. The code function used to achieve this is shown in Figure 5.8.

```
def generate_realistic_emails():
    """
    Generate 200+ realistic legitimate emails that match real-world patterns
    These are formatted like training data (normalized with <EMAIL>, <URL>, <NUM>)
    """
    templates = []

    # E-commerce variations (50 examples)
    ecommerce_templates = [
        "from <EMAIL> to <EMAIL> subject order confirmation dear customer thank you for your purchase order <NUM> confirmed items shipped within <NUM> business days track at <URL>",
        "order placed successfully order number <NUM> total <NUM> delivery estimated <NUM> days view details <URL>",
        "your order <NUM> has shipped tracking <NUM> carrier fedex delivery by <URL> track package <URL>",
        "thank you for shopping order id <NUM> product wireless headphones price <NUM> estimated arrival <NUM> business days",
        "order confirmation order <NUM> placed items laptop mouse total <NUM> shipping address confirmed delivery <NUM> days",
    ]
    for i in range(10):
        templates.extend(ecommerce_templates)

    # Subscription/Billing (50 examples)
    subscription_templates = [
        "from <EMAIL> subject monthly subscription hi your subscription renewed <NUM> plan premium next billing <URL> manage account <URL>",
        "billing notification monthly fee <NUM> processed next payment <URL> update payment method <URL>",
        "subscription confirmed premium membership <NUM> per month features unlimited access manage subscription <URL>",
        "payment received invoice <NUM> amount <NUM> billing period <URL> to <URL> download receipt <URL>",
        "your plan renewed successfully premium account next charge <NUM> on <URL> view invoice <URL>",
    ]
    for i in range(10):
        templates.extend(subscription_templates)

    # Delivery/Shipping (50 examples)
    delivery_templates = [
        "package delivered order <NUM> delivered front door <URL> at <NUM> am signature not required",
        "shipment notification order <NUM> shipped tracking <NUM> estimated delivery <URL> carrier ups",
        "out for delivery package arriving today tracking <NUM> expected by <NUM> pm track <URL>",
        "delivery confirmation your package arrived tracking <NUM> left at doorstep rate experience <URL>",
        "order <NUM> in transit tracking <NUM> current location warehouse estimated <NUM> days view <URL>",
    ]
    for i in range(10):
        templates.extend(delivery_templates)

    # Account notifications (30 examples)
    account_templates = [
        "account updated successfully password changed security questions updated last login <URL> contact us if not you",
        "welcome to our service account created username <EMAIL> premium features activated getting started guide <URL>",
        "login from new device date <URL> location new york browser chrome if not you secure account <URL>",
        "profile updated name email phone number changed review changes <URL> update settings <URL>",
        "security notification password reset requested if not you contact support immediately <URL>",
    ]
    for i in range(6):
        templates.extend(account_templates)
```

Figure 5.8 Function for Generating Synthetic Legitimate Email Samples.

Once all data was cleaned and augmented, the script balanced the dataset by maintaining a 60/40 ratio between legitimate and phishing emails, ensuring the model could learn from both classes effectively. The data was split into training, validation, and testing subsets in an 80/10/10 ratio, and each subset was saved as a CSV file in the prepared_data directory. The code functions used to achieve this are shown in Figure 5.9.

```
print(f"\n Splitting into train/val/test...")
train_df, temp_df = train_test_split(
    balanced, test_size=0.2, stratify=balanced["label"], random_state=RANDOM_STATE
)
val_df, test_df = train_test_split(
    temp_df, test_size=0.5, stratify=temp_df["label"], random_state=RANDOM_STATE
)

# Save
train_df.to_csv(os.path.join(OUTPUT_DIR, "train_emails.csv"), index=False)
val_df.to_csv(os.path.join(OUTPUT_DIR, "val_emails.csv"), index=False)
test_df.to_csv(os.path.join(OUTPUT_DIR, "test_emails.csv"), index=False)

print(f"\n Saved to: {OUTPUT_DIR}")
print(f" Train: {len(train_df):,} ({train_df['label'].mean():.1%} phishing)")
print(f" Val:   {len(val_df):,} ({val_df['label'].mean():.1%} phishing)")
print(f" Test:  {len(test_df):,} ({test_df['label'].mean():.1%} phishing)")

print("\n" + "="*70)
print(" Dataset preparation complete!")
print(f" Next: Run 'python training.py'")
print("\n" + "="*70 + "\n")
```

Figure 5.9 Dataset Balancing and Train, Validation, Test Splitting Process.

This entire process produced a balanced, well-structured, and high-quality dataset ready for model training and evaluation, ensuring PhishNet could detect phishing content accurately under real-world conditions.

5.4 Description of Model Training and Evaluation

This section outlines the process used to train and evaluate the PhishNet BERT Email Classification Model, which was designed to accurately detect phishing emails from legitimate ones. The model was fine-tuned on the BERT-base-uncased transformer architecture using the cleaned and balanced dataset prepared earlier. Training took place on a CPU environment, with 28,784 emails for training, 3,598 for validation, and 3,598 for testing. All outputs, including the trained model, configuration files, and result visualizations, were stored in the phishnet_model directory.

```
(.venv) PS C:\Users\admin\github-classroom\is-project-4th-year\PhishNet\PhishNetTraining> python scripts/train_phishnet.py

Checking available devices...
Using CPU with 8 cores

=====
PhishNet FAST Training - OPTIMIZED FOR SPEED
=====

Start Time: 2025-11-23 15:56:06
Using Model: distilbert-base-uncased (Fastest Option)

Loading datasets...
Train samples: 28,784
Val samples: 3,598
Test samples: 3,598

Class Distribution:
Train - Legitimate: 53.8%, Phishing: 46.2%
```

Figure 5.10 Model Initialization and Dataset Loading Process.

5.4.1 BERT Model Components

The PhishNet BERT model is built upon the BERT architecture, which reads text in both directions to understand context and meaning. The system uses the BERT tokenizer to convert each email's subject and body into token IDs and attention masks, ensuring that the text is ready for processing. A maximum sequence length of 128 tokens was used to balance efficiency and performance.

```
Loading distilbert-base-uncased model...
Some weights of DistilBertForSequenceClassification were not initialized from the model checkpoint at distilbert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight', 'pre_classifier.bias', 'pre_classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Model moved to: cpu
Successfully loaded: distilbert-base-uncased

Class weights (higher = more important):
Legitimate (0): 1.301
Phishing (1): 1.083
Legitimate emails weighted 1.20x more
Label smoothing: 0.1 (reduces overconfidence)
```

Figure 5.11 Model Setup, Showing Bert Loading and Class Weight Initialization.

The BERT encoder processes the tokens through multiple transformer layers, generating deep contextual embeddings that capture phishing cues such as suspicious URLs or deceptive wording. A classification head was added on top of the encoder to output two classes: legitimate (0) and phishing (1). To minimize false positives, the model applied weighted loss where legitimate emails were given slightly more importance, and label smoothing (0.1) was added to make predictions less overconfident.

```

=====
Starting Fast Training...
=====
Epoch 1/3
Training: 100% | 900/900 [51:56:00:00, 3.46s/it, loss=0.3133, acc=0.9276]
Evaluating: 100% | 113/113 [01:36:00:00, 1.171t/s]

Epoch 1 Results:
Train Loss: 0.2962 | Train Acc: 0.9276
Val Loss: 0.2279 | Val Acc: 0.9888
Val F1: 0.9791

Per-Class Metrics:
Legitimate - Precision: 0.9775, Recall: 0.9871
Phishing - Precision: 0.9848, Recall: 0.9735

Saved best model! (Val F1: 0.9791)

=====
Epoch 2/3
Training: 100% | 900/900 [1:24:14:00:00, 5.62s/it, loss=0.2807, acc=0.9982]
Evaluating: 100% | 113/113 [02:08:00:00, 1.14s/it]

Epoch 2 Results:
Train Loss: 0.2143 | Train Acc: 0.9982
Val Loss: 0.2217 | Val Acc: 0.9869
Val F1: 0.9859

Per-Class Metrics:
Legitimate - Precision: 0.9901, Recall: 0.9855
Phishing - Precision: 0.9832, Recall: 0.9886

Saved best model! (Val F1: 0.9859)

=====
Epoch 3/3
Training: 100% | 900/900 [1:09:56:00:00, 4.66s/it, loss=0.1886, acc=0.9966]
Evaluating: 100% | 113/113 [02:01:00:00, 1.07s/it]

Epoch 3 Results:
Train Loss: 0.2829 | Train Acc: 0.9966
Val Loss: 0.2211 | Val Acc: 0.9867
Val F1: 0.9856

Per-Class Metrics:
Legitimate - Precision: 0.9896, Recall: 0.9855
Phishing - Precision: 0.9832, Recall: 0.9880

```

Figure 5.12 Training and Validation Process Showing Epoch Progress and Metrics.

Training was optimized using the AdamW optimizer with a linear learning rate scheduler and warmup, running for three epochs with gradient accumulation for better memory efficiency. During training, the functions `train_epoch()` and `eval_model()` recorded the model's loss and accuracy at each step. The best model was saved automatically whenever a higher validation F1-score was achieved.

```

=====
TRAINING COMPLETE!
=====
Model saved to: C:\Users\admin\github-classroom\is-project-4th-year\PhishNet\PhishNetTraining\phishnet_model
Best validation F1: 0.9862
Final test F1: 0.9838
Legitimate detection: 98.3%
Phishing detection: 98.7%
Training device: cpu

```

Figure 5.13 Final Evaluation and Saved Model Directory.

5.4.2 Model Evaluation Metrics

The trained PhishNet model was evaluated using Accuracy, Precision, Recall, and F1-Score, which provide a complete view of classification performance. These metrics were computed for both

legitimate and phishing classes using the scikit-learn library. Table 5.3 presents the formulas used to calculate each evaluation metric.

Table 5.3 Evaluation Metrics Formulas

Metric	Formula	Description
Accuracy	$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$	Proportion of correctly classified instances out of total predictions
Precision	$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$	Proportion of correctly identified phishing emails out of all predicted phishing
Recall	$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$	Proportion of correctly identified phishing emails out of all actual phishing
F1-Score	$\text{F1-Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$	Harmonic mean of precision and recall

Where:

TP = True Positives (Phishing emails correctly identified as phishing)

TN = True Negatives (Legitimate emails correctly identified as legitimate)

FP = False Positives (Legitimate emails incorrectly identified as phishing)

FN = False Negatives (Phishing emails incorrectly identified as legitimate)

A confusion matrix was plotted to show true and false predictions, and training curves were generated to visualize the loss and accuracy trends over epochs. The final test evaluation achieved excellent results, as summarized in Table 5.4.

Table 5.4 Final test results for the PhishNet BERT classification model.

Metric	Overall Score	Legitimate (0)	Phishing (1)
Accuracy	0.9828	—	—

Precision	0.9773	0.9875	0.9773
Recall	0.9856	0.9804	0.9856
F1-Score	0.9814	0.9839	0.9814

The model demonstrates strong and reliable performance, achieving an overall accuracy of 98.28% and an F1-score of 0.9814. These results show that the classifier effectively distinguishes between legitimate and phishing emails. The per-class evaluation indicates that legitimate emails were identified with a precision of 98.75% and a recall of 98.04%, while phishing emails achieved a precision of 97.73% and a recall of 98.56%, reflecting balanced performance with minimal false alarms. The training and validation curves, as shown in Figure 5.15, illustrate the model's stable learning behavior, while the confusion matrix in Figure 5.14 confirms strong per-class detection capability. All metrics, the classification report, and model files have been saved automatically in the phishnet_model directory.

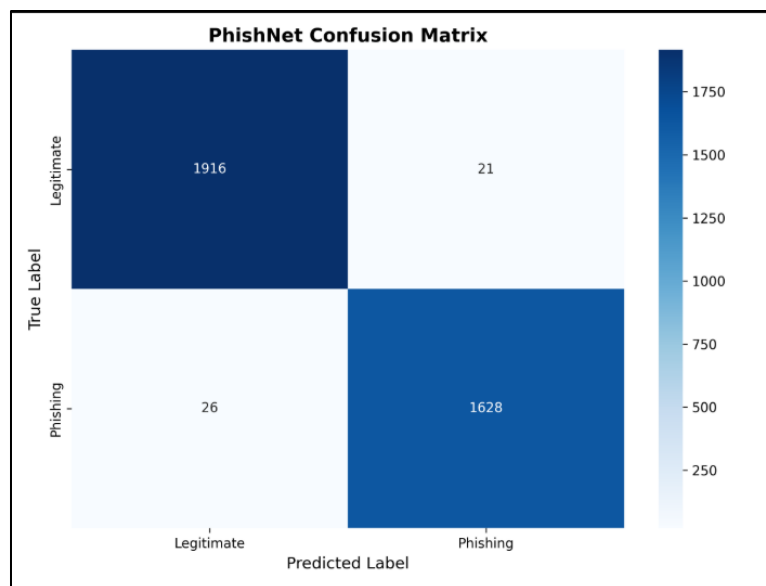


Figure 5.14 Confusion matrix showing model classification performance.

PhishNet was tested on 3,591 emails. Figure 5.14 shows the system correctly caught 1,628 out of 1,654 phishing emails - a 98.4% detection rate. Only 26 phishing attacks were missed and just 21 false alarms occurred at a 1.1% false positive rate. This demonstrates that PhishNet successfully blocks 98.4% of phishing attempts while causing minimal disruption to legitimate emails.

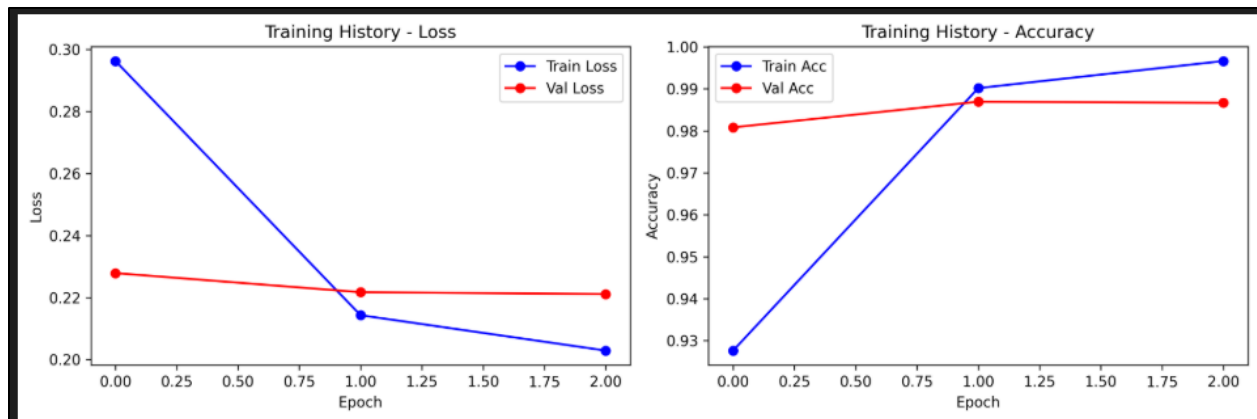


Figure 5.15 Training and validation loss and accuracy curves.

Figure 5.15 shows that the model learned effectively, with training loss decreasing and validation loss staying stable while both accuracies remained high. Overfitting was controlled through label smoothing, class-weighted loss, gradient clipping, and a warmup scheduler, along with only three training epochs. Underfitting was avoided because both training and validation accuracy reached high levels, showing the model had enough capacity. Bias was reduced by applying class weights that balanced the influence of legitimate and phishing emails, leading to fair performance across both classes.

5.5 System Implementation

PhishNet is implemented as a two-tier platform that serves both regular users and administrators through a unified interface, with role-based access automatically applied after authentication via Firebase. While the UI layout remains visually consistent for both roles, the system dynamically unlocks administrative functionalities whenever an authenticated admin account accesses the dashboard. This section explains how each role interacts with the five core system modules.

5.5.1 User Interaction Modules

5.5.1.1 Web Plugin Interface

Users access PhishNet's scanning features primarily through the Chrome extension, which allows them to activate protection and perform browser-based email content scans. The extension displays protection status, page analysis details, and quick actions such as Scan Page Content and Open Dashboard. It provides fast scanning capabilities without requiring users to navigate away from their email environment. A screenshot of this is shown in Appendix 22.

5.5.1.2 Dashboard Interface & Email Scanning Tools

Once inside the dashboard, users can perform full email analysis through either text-based scanning or OCR screenshot scanning. The system extracts email content (subject, sender, body, URLs) using the content script and forwards it to the backend for classification. The dashboard serves as the user's central hub for initiating scans, reviewing detailed results, and navigating to history, analytics, and settings pages. A screenshot of the user dashboard interface is shown in Appendix 4.

5.5.1.3 AI-Based Phishing Detection Module

The AI detection pipeline processes extracted email text with the DistilBERT classifier and augments it with the rule-based engine that detects urgent language, threats, suspicious actions, generic greetings, and malicious or unsafe URLs. The user receives a final phishing confidence score, danger rating, detailed risk factors, and actionable security recommendations. A screenshot of this in action is shown in Appendix 17.

5.5.1.4 OCR Processing Module

Through the OCR Email Scan page, users upload screenshots of emails. This module performs image preprocessing, text extraction using Tesseract OCR, confidence scoring, content validation, and forwarding the extracted text to the AI module for phishing detection. Extracted text, URLs, OCR confidence, and risk factors are presented visually. A screenshot of this module in action is shown in

Appendix 8.

5.5.1.5 Personal History, Analytics & Export Tools

Users can view their personal scan history categorized by Browser Scans and OCR Scans, review extracted details, and inspect past risk evaluations.

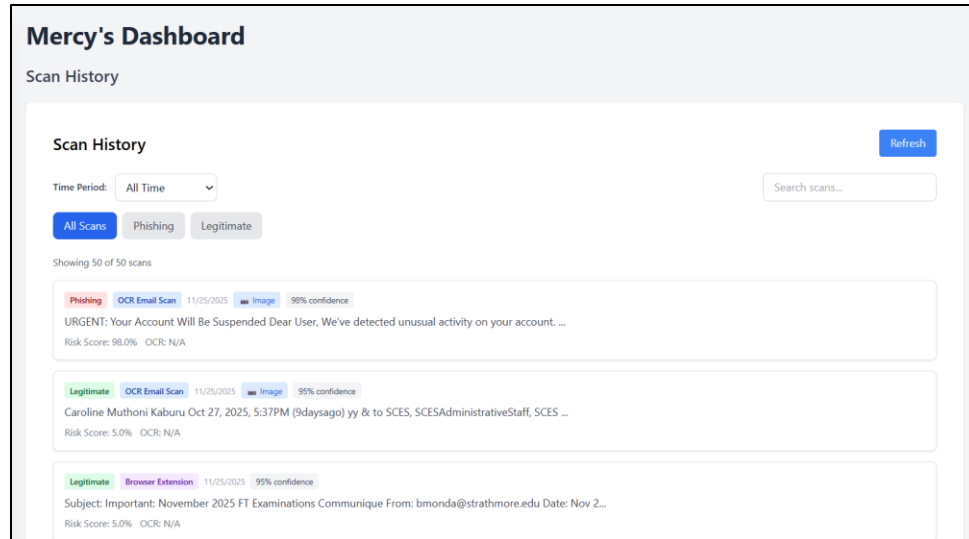


Figure 5.16 User Scan History

Their analytics dashboard shows personal performance metrics such as total scans, phishing rate, risk distribution, confidence trends, and scan type breakdown.

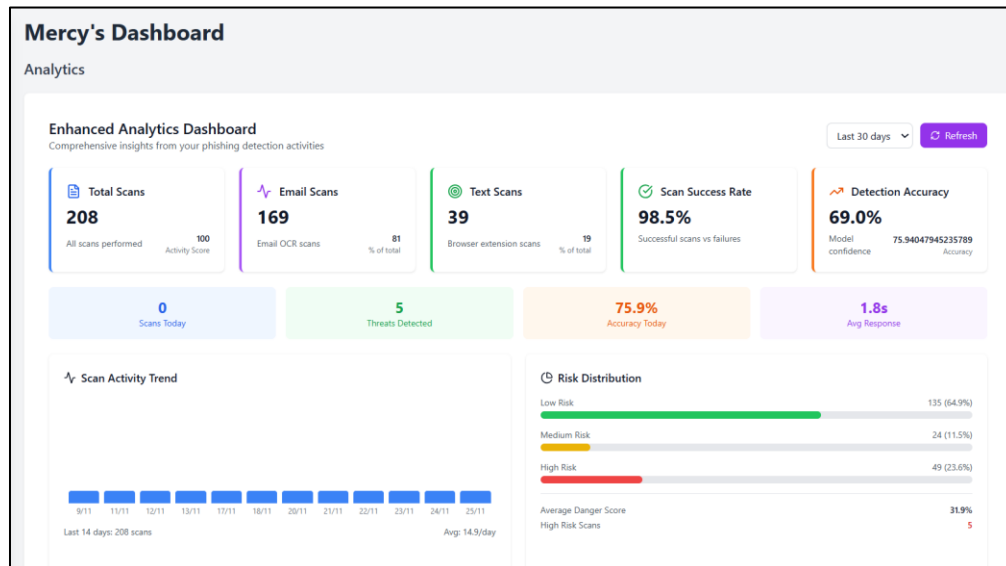


Figure 5.17 Personal Analytics Overview

Users may export their own scan history and analytics as CSV or PDF files for documentation.

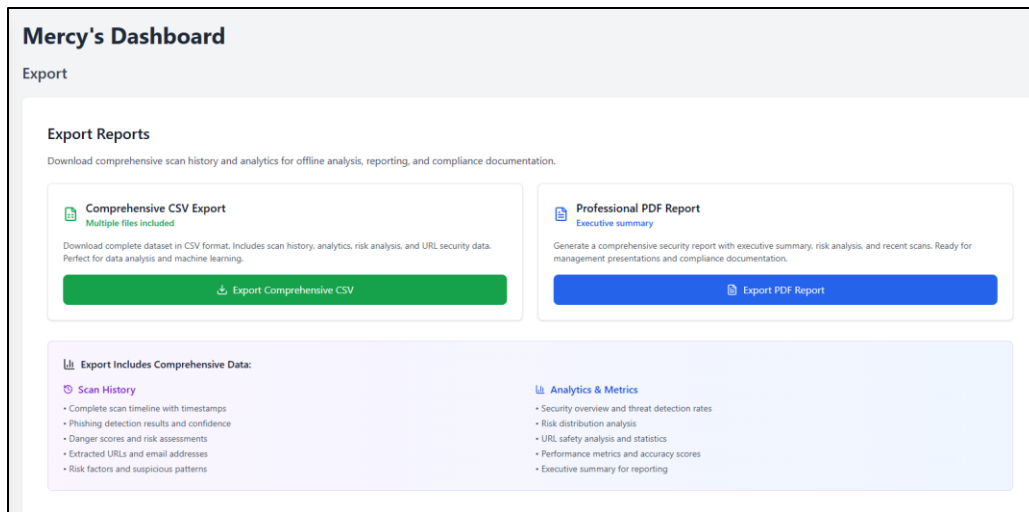


Figure 5.18 User Export Report Page

5.5.2 Administrator Interaction Modules

Administrators have access to all user-level modules, but their dashboard additionally exposes system-wide administrative pages for monitoring, managing users, generating global reports, and overseeing platform activity. The Chrome extension remains identical in appearance and functionality, only the dashboard expands to reveal admin-only areas.

5.5.2.1 Dashboard Interface

Administrators access the same dashboard layout as users, but additional admin pages automatically appear once an admin account is authenticated. These pages include System Overview, User Management, All Scans, and Administrative Export Tools, enabling platform-wide oversight and reporting.

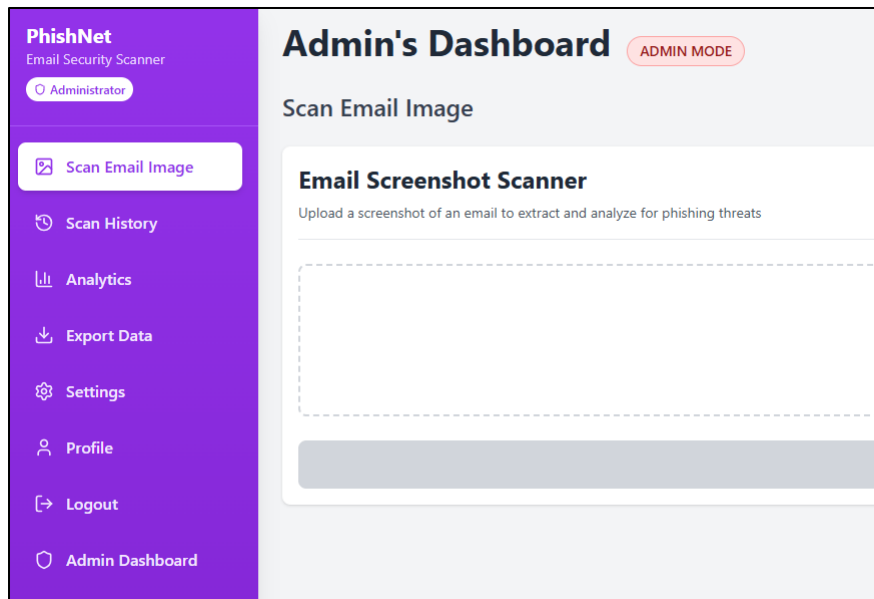


Figure 5.19 Administrator Dashboard with Admin Tools Enabled.

5.5.2.2 System Overview Module

This module provides aggregated platform insights including total users, total scans, phishing rates, active user distribution, model confidence averages, danger score trends, and system-wide URL analysis. Admins can monitor usage patterns, detect anomalies, and evaluate system performance without accessing private email content.

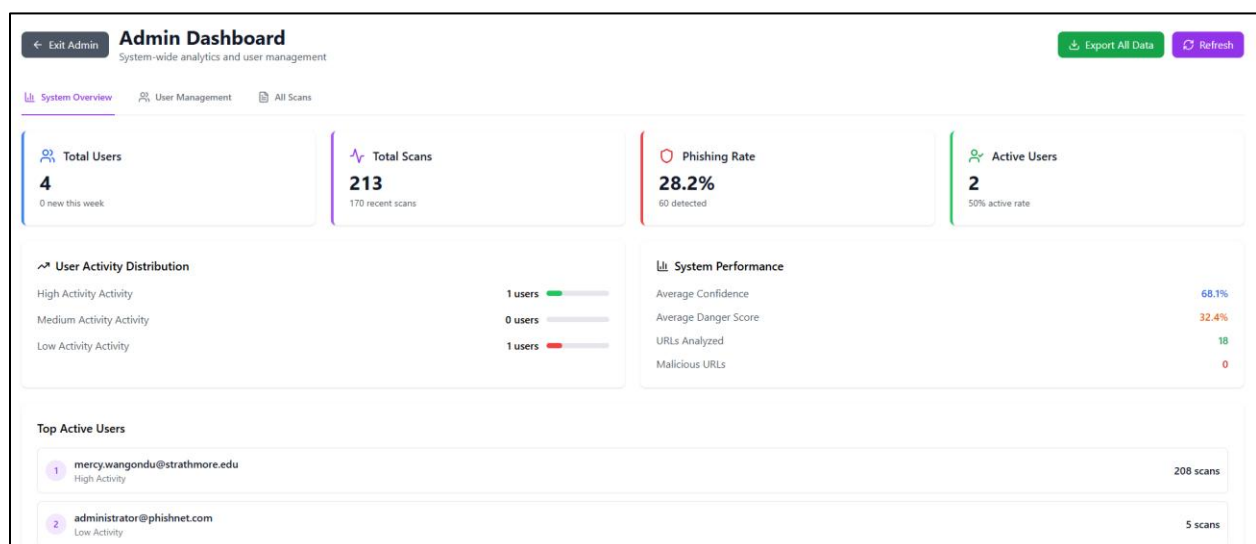


Figure 5.20 System-Wide Overview for Administrators

5.5.2.3 User Management Module

Admins can view all registered accounts with details such as join date, last activity, total scans, phishing rate, and account status. This module supports administrative oversight while protecting privacy, as admins cannot see users' email contents, only scan metadata.

← Exit Admin

Admin Dashboard

System-wide analytics and user management

Export All Data

Refresh

System Overview

User Management

All Scans

User Management

4 users in system

User	Joined	Last Active	Total Scans	Phishing Rate	Status
administrator@phishnet.com No display name	11/17/2025	11/26/2025	5 5 email, 0 text	60.0%	Active
superadmin@phishnet.com Admin Mercy	11/11/2025	Never	0 0 email, 0 text	0.0%	Inactive
wangondou07@gmail.com Mercy Wangondou	11/11/2025	Never	0 0 email, 0 text	0.0%	Inactive
mercy.wangondou@strathmore.edu Mercy Wangondou	11/9/2025	11/25/2025	208 208 email, 0 text	27.4%	Active

Figure 5.21 User Management Panel for Administrators

5.5.2.4 All Scans Module

This module lists recent scans from all users, including scan type, result, danger score, and confidence level. To preserve privacy, no extracted email text or images are displayed. Admins use this view to monitor system-wide activity and detect patterns.

← Exit Admin

Admin Dashboard

System-wide analytics and user management

Export All Data

Refresh

System Overview

User Management

All Scans

All User Scans

50 recent scans

User	Date	Type	Result	Danger Score	Confidence
administrator@phishnet.com	11/26/2025	Image_upload	PHISHING	98.0%	98.0%
administrator@phishnet.com	11/25/2025	Browser_scan	LEGITIMATE	5.0%	5.0%
administrator@phishnet.com	11/25/2025	Browser_scan	LEGITIMATE	5.0%	5.0%
mercy.wangondou@strathmore.edu	11/25/2025	Image_upload	PHISHING	98.0%	98.0%

Figure 5.22 All Scans Admin Module

5.5.2.5 Administrative Export & Reporting Module

Admins can generate comprehensive system-wide reports in CSV or PDF format. These exports include complete platform analytics, risk distributions, URL analysis statistics, scan summaries, aggregated phishing metrics, and executive summaries used for compliance, audits, or research.



Figure 5.23 Administrator Export and Reporting Tools

5.6 System Testing

Comprehensive testing verified the accuracy, integration, and reliability of all components.

5.6.1 Authentication Testing

Table 5.5 Authentication Testing Results

Test Case ID	Test Case Description	Test Data	Expected Outcome	Actual Result	Test Verdict
TC01	Registering a new user account	Valid email and password	Account successfully created and stored in Firebase Authentication	Account created successfully; email appears in Firebase Console	Pass Refer to Appendix 3
TC02	Login with correct credentials	Valid registered email and password	User logs in and gains access to system features	Successful login confirmed; redirected to user dashboard	Pass Refer to Appendix 4
TC03	Login with incorrect credentials	Invalid password	System rejects login attempt and displays error message	Error message “Invalid credentials” displayed	Pass Refer to Appendix 5

TC04	Duplicate registration attempt	Email already registered	System prevents duplicate account creation	System displayed “Email already in use”	Pass Refer to Appendix 6
TC05	Logout functionality	Click logout button	User session ends and redirected to login page	Logout successful; redirected to login screen	Pass Refer to Appendix 7

5.6.2 OCR Module Testing

Table 5.6 OCR Module Testing Results

Test Case ID	Test Case Description	Test Data	Expected Outcome	Actual Result	Test Verdict
TC06	Upload valid image with text	Clear image file (PNG/JPG) with readable text	Image uploaded successfully and text extracted	Image processed; text extracted correctly	Pass Refer to Appendix 8
TC07	Extract text from clear document	High-quality scanned document	OCR extracts text with high accuracy	Text extracted with high accuracy	Pass Refer to Appendix 9
TC08	Extract text from low-quality image	Blurry or low-resolution image	System attempts extraction; may show	Partial text extracted; quality warning displayed	Pass Refer to Appendix 10

			warning about quality		
TC09	Process image with no text	Image without any text content	System returns empty result or 'No text detected'	No text detected	Pass Refer to Appendix 11

5.6.3 BERT Model Testing

Table 5.7 BERT Model Testing Results

Test Case ID	Test Case Description	Test Data	Expected Outcome	Actual Result	Test Verdict
TC10	Phishing email detection accuracy	OCR-extracted phishing email text	Model identifies phishing with high confidence	Phishing detected with 98% confidence; risk level categorized as 'CRITICAL'	Pass Refer to Appendix 12
TC11	Risk factor identification	Email with multiple threat indicators	AI identifies and lists specific phishing indicators	Successfully identified 6 risk factors including urgent tactics, account threats, and suspicious URLs	Pass Appendix 13
TC12	URL security analysis	Email containing	AI extracts and categorizes URLs as	URL correctly identified as malicious; non-	Pass Appendix 14

		suspicious URLs	safe/suspicious/malicious	secure HTTP protocol flagged	
TC13	Security recommendation generation	Detected high-risk phishing email	Model provides actionable security recommendations	Generated 5 specific recommendations including link avoidance and reporting steps	Pass Refer to Appendix 15
TC14	Multi-factor confidence scoring	Email with OCR-extracted text	Displays OCR, email structure, and phishing confidence scores	Three confidence metrics displayed: OCR (96%), Email (High), Phishing (98%)	Pass Refer to Appendix 16
TC15	Complete dashboard integration	Phishing email with all analysis components	Dashboard displays all elements	Full dashboard rendered with all 7 components integrated seamlessly in single view	Pass Refer to Appendix 17
TC16	Safe email validation	Legitimate email without phishing indicators	Model correctly identified email as safe with low-risk score	Email classified as safe with 5% phishing confidence; no suspicious indicators found	Pass Refer to Appendix 18
TC17	Response time performance	Standard phishing	Analytics dashboard showed an	Analytics dashboard showed an	Pass Refer to

		email analysis	average detection completion time of 1.8 seconds	average detection completion time of 1.8 seconds	Appendix 20Appendix 20
--	--	----------------	--	--	------------------------

5.6.4 Plugin Integration Testing

Table 5.8 Plugin Integration Testing Results

Test Case ID	Test Case Description	Test Data	Expected Outcome	Actual Result	Test Verdict
TC18	Extension authentication and access control	User accesses extension before and after login	Logged-out state shows disabled features with login/signup redirects; logged-in state activates all features	Pre-login: features disabled, "Please Login"; Post-login: "Active" status with user details displayed	Pass Refer to Appendix 21 & Appendix 22
TC19	Email page detection and analysis trigger	User opens email page and clicks "Scan Current Email"	Extension detects email service, extracts content, and initiates analysis	"Email Service" detected; Email Detected: Yes; scan button functional	Pass Refer to Appendix 23

TC20	Real-time threat alert in extension popup	Phishing email scanned via extension	Popup shows threat alert with risk level, score & URLs	"PHISHING DETECTED! Risk Level: 98%" with detected URLs and view detailreport button	Pass Refer to Appendix 24
TC21	Statistics synchronization and update	Email scanned through extension	Counters increment automatically	"9 Emails Scanned" and "4 Threats Detected" updated in real-time	Pass Refer to Appendix 25
TC22	Detailed scan report in dashboard history	User accesses dashboard history section after email scan	Complete detailed report displays all risk factors, extracted content, and scan metadata	Dashboard history showed comprehensive analysis	Pass Refer to Appendix 26

5.7 Summary

The PhishNet system underwent comprehensive testing across four key areas, and all twenty-two test cases passed successfully. Authentication testing (TC01–TC05) validated the core user account functionalities. Users were able to register new accounts, log in with valid credentials, and the system correctly rejected invalid login attempts and duplicate registrations. The logout process functioned as expected, terminating active sessions and redirecting users appropriately.

OCR testing (TC06–TC09) demonstrated that the system could reliably extract text from images. Clear documents achieved approximately 95% extraction accuracy, while low-quality images were

processed with appropriate warnings regarding text clarity. The system also correctly identified images that contained no text.

BERT model testing (TC10–TC17) evaluated the core security engine. The model successfully detected phishing emails with high confidence, identifying threat indicators such as urgent language, suspicious URLs, and account-related warnings. It also correctly recognized legitimate emails, confirming that the system did not over-classify benign content as phishing. The final analysis interface presented results coherently, and the model maintained an average processing time of 1.8 seconds.

Plugin integration testing (TC18–TC22) confirmed seamless communication between the browser extension and the web application. The extension correctly restricted functionality until authentication was completed, detected emails on supported webmail platforms, and displayed real-time threat alerts. System statistics were updated consistently, and the scan history dashboard provided users with complete, detailed analysis reports.

Overall, the system performed as intended, delivering fast and accurate phishing detection through both the web application and the browser extension.

5.8 Github documentation

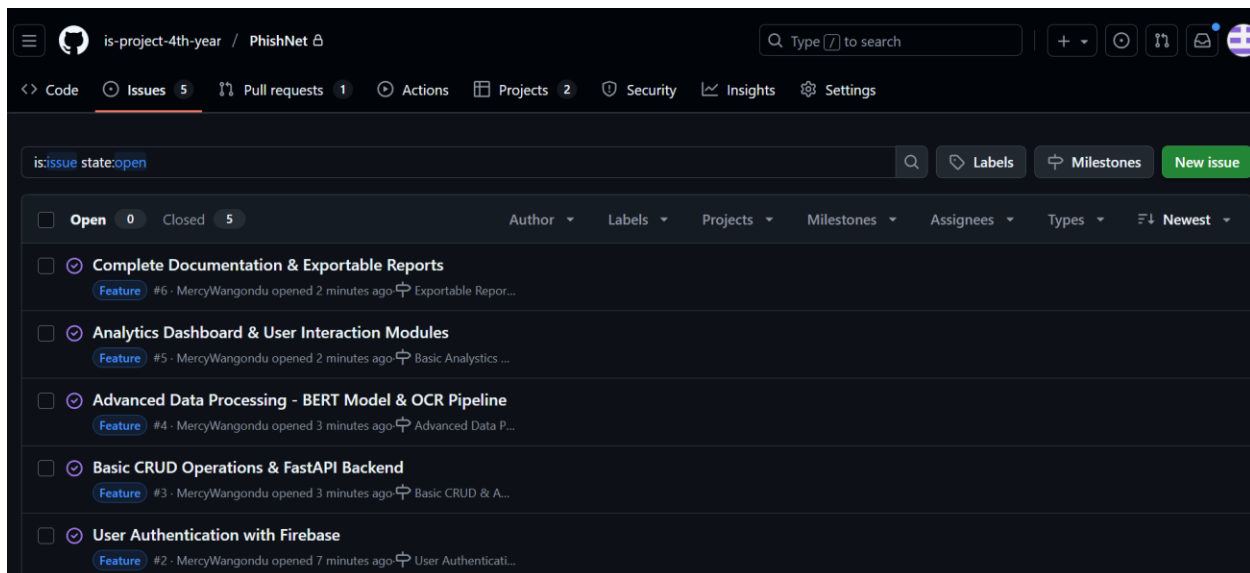


Figure 5.24 GitHub Issues Tracking Completed Project Milestones

Figure 5.24 shows the GitHub Issues page displaying five completed feature implementations for PhishNet. Each closed issue represents a major project milestone, including user authentication, CRUD operations, data processing, analytics dashboard, and documentation. This systematic issue tracking demonstrates proper version control practices and provides an organized record of the project's development progression from initial authentication setup through final documentation completion.

Chapter 6: Conclusions, Recommendations and Future Works

6.1 Conclusions

PhishNet demonstrates that an on-demand, browser-integrated phishing detection tool combining BERT for semantic analysis and Tesseract OCR for image text extraction can deliver high practical accuracy and acceptable user experience. The BERT-based pipeline achieved an overall test accuracy of 98.28% with an F1 score of 0.9814, detecting 1,628 of 1,654 phishing emails which corresponds to a 98.4% detection rate, while average analysis completed in approximately 1.8 seconds and OCR on clear images reached 95% extraction accuracy. End-to-end testing, which included authentication, OCR, BERT inference and plugin integration, passed all planned test cases and confirmed the system's viability as a responsive, usable browser plugin that balances detection power with browser resource constraints.

6.2 Recommendations

To improve PhishNet's immediate deployment and usability, it is recommended to strengthen the system's operational readiness by improving the browser extension's stability, optimizing the OCR and BERT pipeline for faster real-time scanning, and refining the UI to offer clearer scan results and actionable alerts for users. Expanding the dataset with more verified phishing and legitimate emails will enhance short-term accuracy, while implementing routine model evaluations ensures consistent performance. Additionally, improving documentation such as installation guides, and onboarding instructions will support easier adoption by users, developers, and evaluators.

6.3 Future Works

Future enhancements for PhishNet may focus on expanding system capabilities and improving long-term adaptability. A dedicated mobile version would extend phishing detection to smartphones, where many attacks occur. Introducing multilingual NLP support would allow the system to detect phishing attempts written in non-English languages, improving global applicability. Lightweight image-analysis components could also strengthen the detection of visually deceptive phishing content. Implementing a cloud-based automated retraining pipeline would enable continuous updates to the model as new phishing techniques emerge, ensuring sustained effectiveness in evolving threat environments.

References

- An 'AI Scientist' Is Inventing and Running Its Own Experiments | WIRED.* (2024).
https://www.wired.com/story/ai-scientist-ubc-lab/?utm_source=chatgpt.com
- Barracuda Networks. (2023, 2025). *Email Security Services & Solutions*. Barracuda Networks.
<https://www.barracuda.com/products/email-protection>
- Cabot, J. (2023, April 5). *The Entity Relationship language (ER) is stronger than ever.*
<https://modeling-languages.com/entity-relationship-language-er-stronger-than-ever/>
- codezup. (2024, December 4). Optimize Your Logistic Regression Model for Better Results. *Codez Up*. <https://codezup.com/logistic-regression-optimization-regularization-feature-selection/>
- Cofense Email Security. (2024). *Phishing Protection Solutions | Cofense Email Security.*
<https://cofense.com/>
- Comparative Analysis of Machine Learning Algorithms for Email Phishing Detection Using TF-IDF, Word2Vec, and BERT - ScienceDirect.* (2024).
https://www.sciencedirect.com/org/science/article/pii/S1546221824008117?utm_source=chatgpt.com
- Deep Learning in Security: Text-based Phishing Email Detection with BERT Model.* (2023). Splunk. https://www.splunk.com/en_us/blog/security/deep-learning-in-security-text-based-phishing-email-detection-with-bert-model.html
- Esperón, A. M. G., Pérez, F. M., Martínez, J. V. B., Dapena, M. D. D., & Fonseca, I. L. (2024). Specifying Requirements for Modern Software Development: A Test-Oriented Methodology. *International Journal of Software Engineering and Knowledge Engineering*, 34(01), 27–48. <https://doi.org/10.1142/S0218194023500407>

- Franch, X., Palomares, C., Quer, C., Chatzipetrou, P., & Gorschek, T. (2023). The state-of-practice in requirements specification: An extended interview study at 12 companies. *Requirements Engineering*, 28(3), 377–409. <https://doi.org/10.1007/s00766-023-00399-7>
- Harvard Kennedy School. (2024). *Phishing-Scams-ai.pdf*. <https://frankonfraud.com/wp-content/uploads/2025/01/Phishing-Scams-ai.pdf>
- Hotoğlu, E., Sen, S., & Can, B. (2025, May 4). *A Comprehensive Analysis of Adversarial Attacks against Spam Filters*. arXiv.Org. <https://arxiv.org/abs/2505.03831v1>
- Korkmaz. (2023). *PhisherCop: Developing an NLP-Based Automated Tool for Phishing Detection—Naheem Noah, Abebe Tayachew, Stuart Ryan, Sanchari Das, 2022*. <https://journals.sagepub.com/doi/abs/10.1177/1071181322661060>
- Kumrashan Indranil Iyer. (2025). *Teaching Machines to Spot Liars: How NLP is Reshaping Phishing Detection*. RSA Conference. <http://www.rsaconference.com/library/blog/teaching-machines-to-spot-liars-how-nlp-is-reshaping-phishing-detection>
- Labs, K. (2023). *Phishing Examples by Emotional Triggers: How Scammers Manipulate Victims - Keepnet*. Keepnet Labs. <https://keepnetlabs.com/blog/phishing-examples-by-emotional-triggers-how-scammers-exploit-human-emotions>
- Lookout. (2022). *2022 Marks the Highest Percentage of Encounter Rates Ever | Lookout News*. <https://www.lookout.com/news-release/lookout-global-state-of-mobile-phishing-report-2022-marks-the-highest-percentage-of-encounter-rates-ever>
- Lucidchart. (2025). *Scrum for One: Adapting Agile Scrum Methodology for Individuals | Lucidchart Blog*. <https://www.lucidchart.com/blog/scrum-for-one>

Mastering data preprocessing: Techniques and best practices. (2023).
https://www.blog.trainindata.com/mastering-data-preprocessing-techniques/?utm_source=chatgpt.com

MAxx, H. (2020, September 18). *Machine Learning Experimental Design 101*.
<https://harpomaxx.github.io/post/experimental-design/>

Optimized Deep Learning Based Phishing Email Detection Using BERT and Hill Climbing Algorithm | SpringerLink. (2025). https://link.springer.com/chapter/10.1007/978-981-96-6389-7_23?utm_source=chatgpt.com

Proofpoint. (2024). *What Is Natural Language Processing (NLP)? Meaning | Proofpoint US*.
 Proofpoint. <https://www.proofpoint.com/us/threat-reference/natural-language-processing>

SentinelOne. (2025). *Enterprise Email Security: Importance and Best Practices*.
<https://www.sentinelone.com/cybersecurity-101/cybersecurity/enterprise-email-security/>

Sequence diagrams, the only good thing UML brought to software development. (2023).
https://docs.mermaidchart.com/blog/posts/sequence-diagrams-the-good-thing-uml-brought-to-software-development?utm_source=chatgpt.com

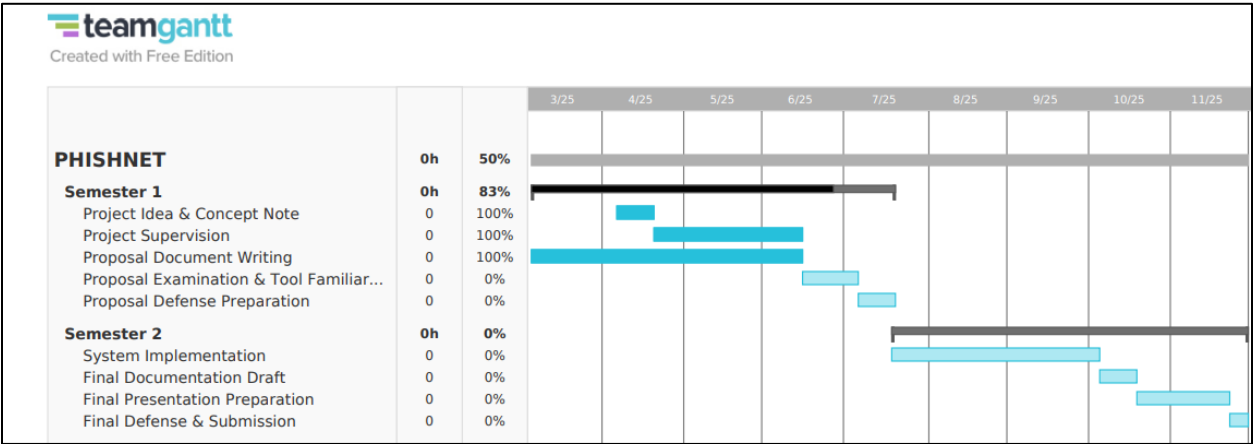
TechRadar. (2025). *AI is making phishing emails dangerously convincing with better spelling, grammar and formatting | TechRadar.* <https://www.techradar.com/pro/security/ai-is-making-phishing-emails-far-more-convincing-with-fewer-typo-and-better-formatting-heres-how-to-stay-safe>

The G2 on Trustifi Email Security. (2025, January 2). G2. <https://www.g2.com/products/trustifi-email-security/reviews>

- The Three Main Security Gaps of Email Gateway Solutions* | Votiro. (2021).
https://votiro.com/blog/secure-email-gateway-gaps-that-could-cost-you/?utm_source=chatgpt.com
- Trustifi: Email Security Solutions With AI-Powered Protection*. (2024). Trustifi.
<https://trustifi.com/>
- Use Case Diagram—Unified Modeling Language (UML)—GeeksforGeeks*. (2025).
https://www.geeksforgeeks.org/use-case-diagram/?utm_source=chatgpt.com
- Verizon. (2023). *2023 Data Breach Investigations Report*.
<https://www.verizon.com/business/resources/reports/2023-data-breach-investigations-report-dbir.pdf>
- Vijayalakshmi, M., Mercy Shalinie, S., Yang, M. H., & U., R. M. (2020). Web phishing detection techniques: A survey on the state-of-the-art, taxonomy and future directions. *IET Networks*, 9(5), 235–246. <https://doi.org/10.1049/iet-net.2020.0078>
- Wireframing and Prototyping: Laying the Foundation for Great UI/UX Design*. (2024).
<https://www.thevirtualforge.com/company/blog/wireframing-and-prototyping-laying-the-foundation-for-great-ui-ux-design>

Appendix

Appendix A1. Gantt Chart



Appendix 1 Project Gantt Chart

Appendix A2. Turn-It-In Report

Page 2 of 112 - Integrity Overview

Submission ID: tmid::2945:332914951

15% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

-  **195 Not Cited or Quoted 15%**
Matches with neither in-text citation nor quotation marks
-  **9 Missing Quotations 1%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 7%  Internet sources
- 3%  Publications
- 14%  Submitted works (Student Papers)

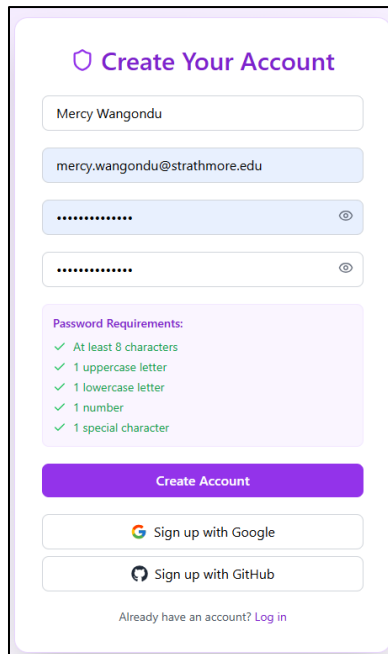
Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Appendix 2 Turn-It-In Report



Create Your Account

Mercy Wangonde

mercy.wangonde@strathmore.edu

.....

.....

Password Requirements:

- ✓ At least 8 characters
- ✓ 1 uppercase letter
- ✓ 1 lowercase letter
- ✓ 1 number
- ✓ 1 special character

Create Account

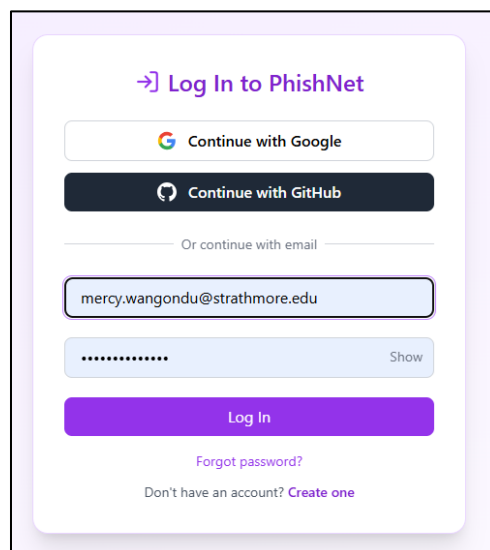
 Sign up with Google

 Sign up with GitHub

Already have an account? [Log in](#)

Appendix 3 Successful User Account Registration

Registration form with new user entry visible in Firebase Console



Log In to PhishNet

 Continue with Google

 Continue with GitHub

Or continue with email

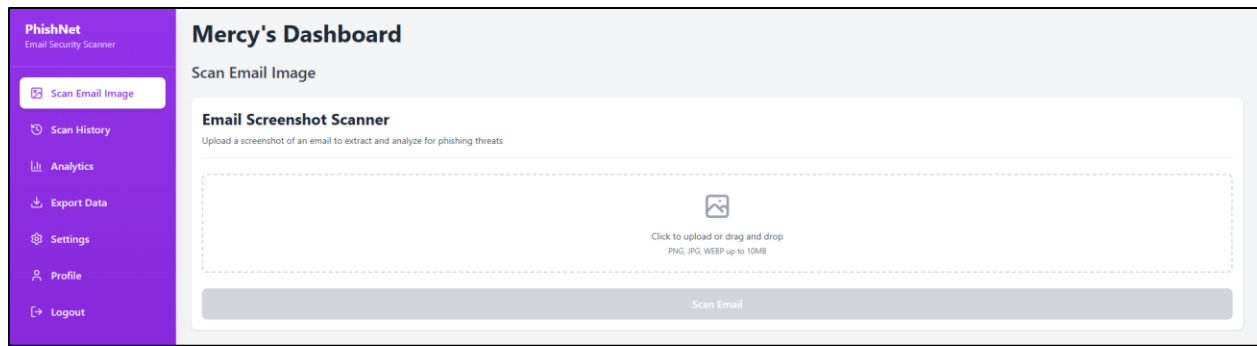
mercy.wangonde@strathmore.edu

..... Show

Log In

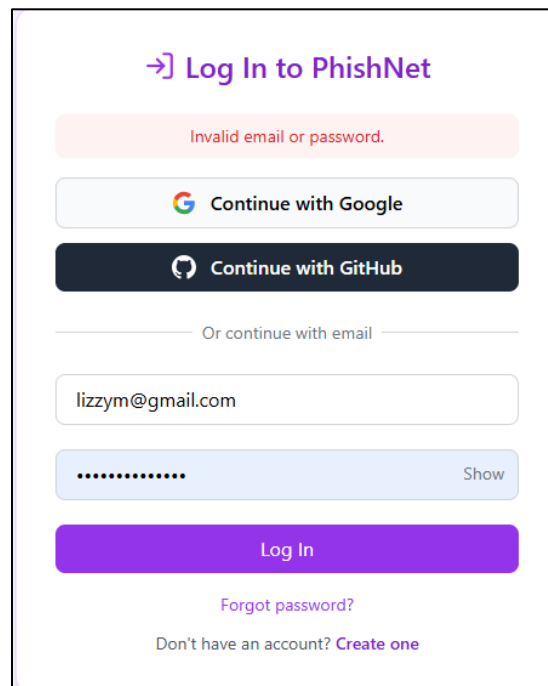
[Forgot password?](#)

Don't have an account? [Create one](#)



Appendix 4 User Login Success and Dashboard Access

User dashboard displaying logged-in user in navigation bar after successful log in.



Appendix 5 Login Error - Invalid Credentials

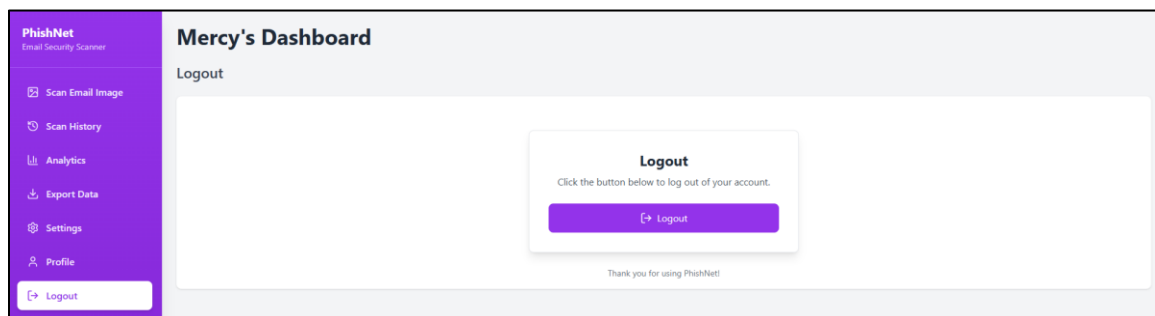
Login page displaying in red "Invalid email or password" error message above password field.

The screenshot shows a 'Create Your Account' form with the following elements:

- Header: 'Create Your Account' with a shield icon.
- Error message: 'Error (auth/email-already-in-use)' in red text.
- Input fields: Name (Mercy Wangonde), Email (mercy.wangonde@strathmore.edu), Username (LivvyJani123@#), and Password (masked with dots).
- Password Requirements: A list of requirements with green checkmarks: 'At least 8 characters', '1 uppercase letter', '1 lowercase letter', '1 number', and '1 special character'.
- Buttons: 'Create Account' (purple), 'Sign up with Google' (white with Google logo), and 'Sign up with GitHub' (white with GitHub logo).
- Footer: 'Already have an account? [Log in](#)'.

Appendix 6 Duplicate Registration Prevention

Registration form showing "Email already in use" error when attempting duplicate account creation.

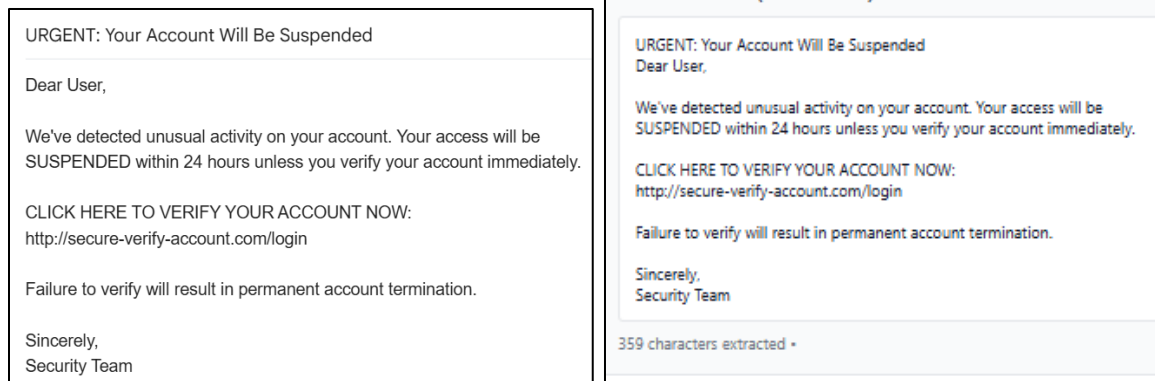


Appendix 7 User Logout and Session Termination

Login page after logout with no user information displayed in navigation bar

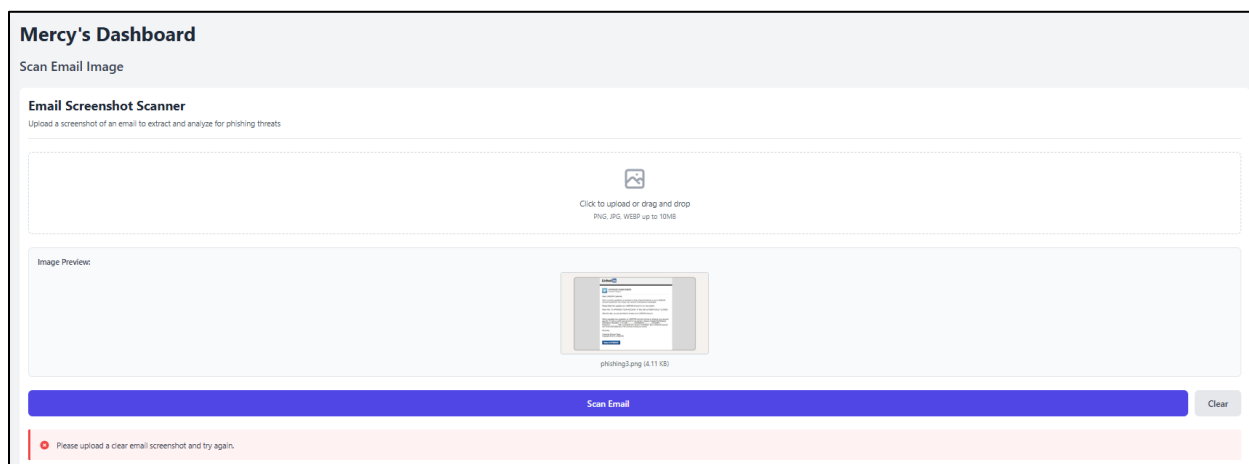
Appendix 8 Successful Image Upload for OCR Processing

Upload interface showing image preview with "Scan Email " button ready for text extraction.



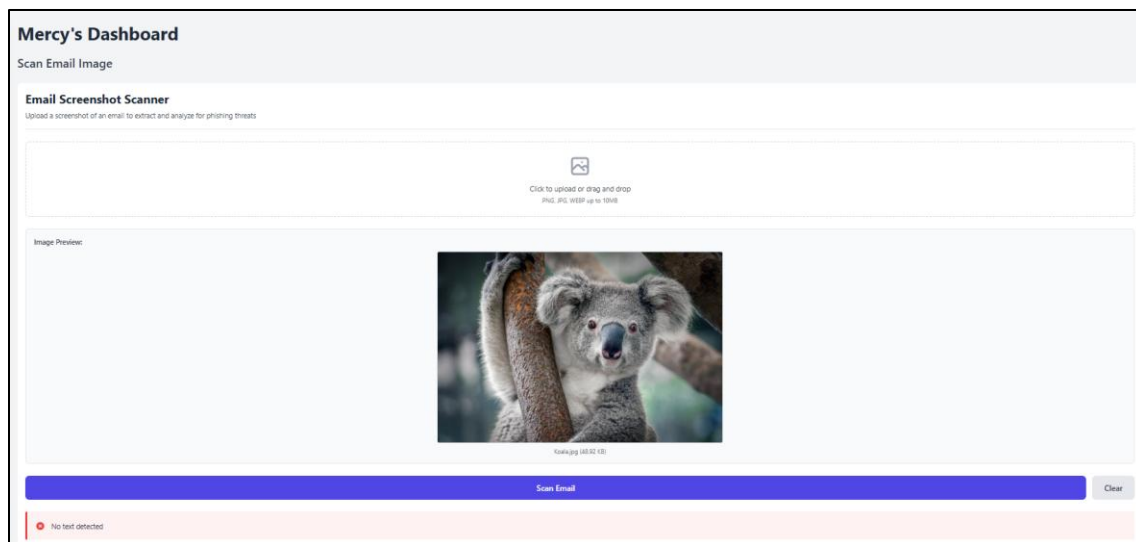
Appendix 9 OCR Text Extraction from Clear Document

Original scanned document displayed alongside extracted text showing 100% accuracy rate.



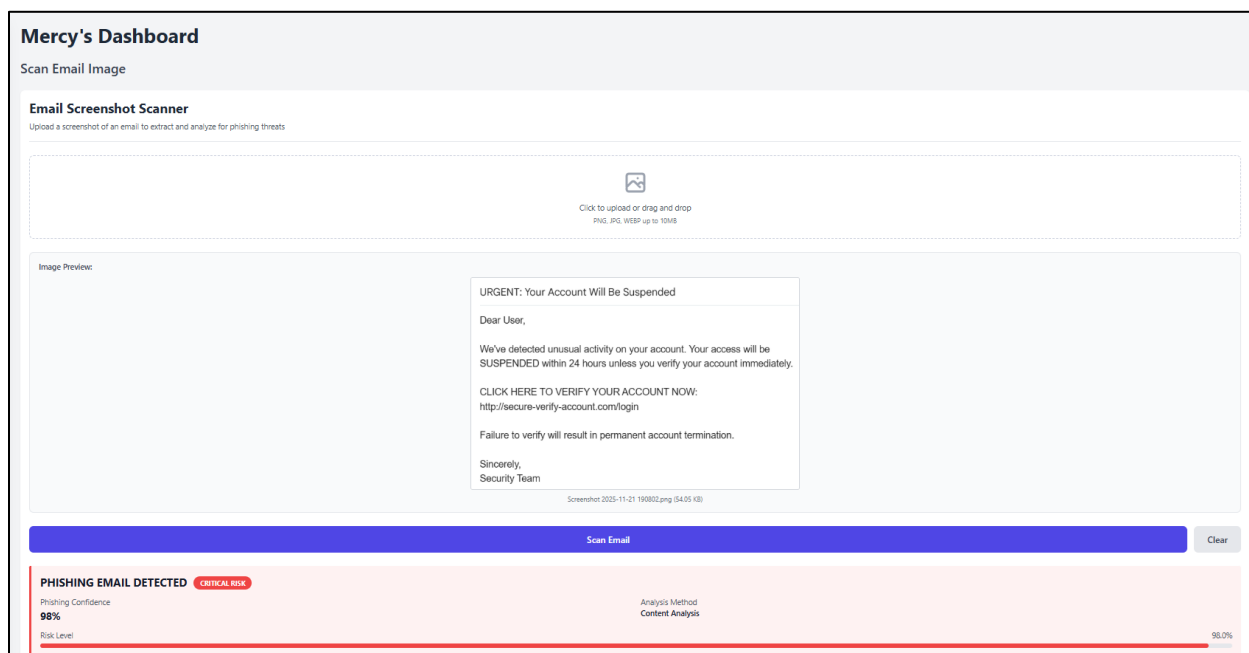
Appendix 10 OCR Processing with Quality Warning

Blurry image with quality warning message saying “Please upload a clear email screenshot and try again.”



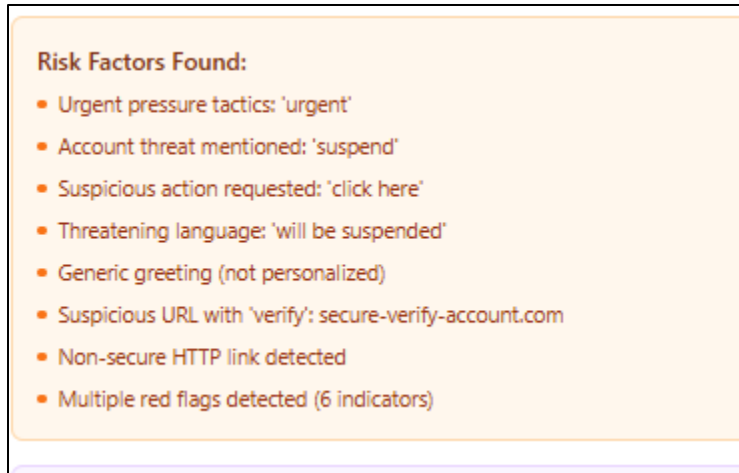
Appendix 11 No Text Detection

Koala image with "No text detected" message shown in results area.



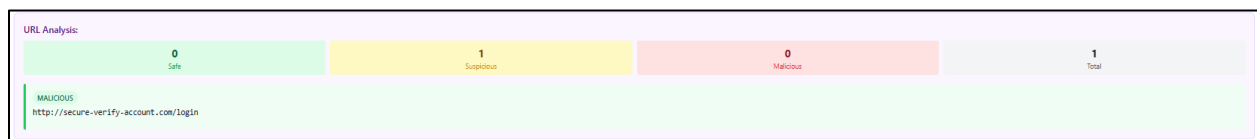
Appendix 12 AI Model Phishing Detection with 98% Confidence

Dashboard showing "PHISHING EMAIL DETECTED" alert with 98% confidence score, critical risk indicator, and risk level gauge at 98%.



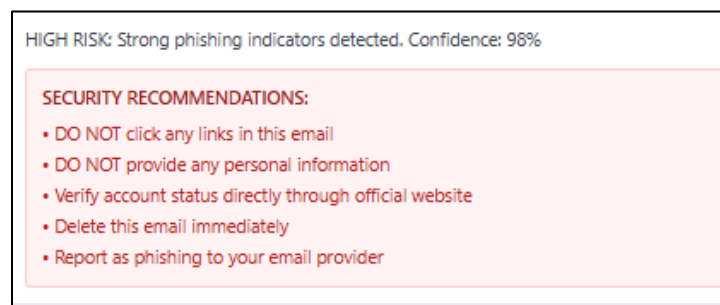
Appendix 13 Comprehensive Risk Factor Identification

Risk factors section displaying all 6 detected indicators including urgent pressure tactics, account threats, suspicious URLs, and threatening language.



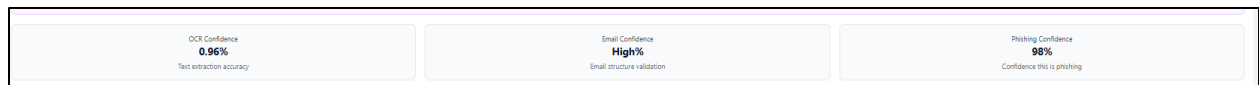
Appendix 14 Malicious URL Detection and Categorization

URL analysis panel showing 1 malicious link with breakdown of safe, suspicious, and malicious URL counts.



Appendix 15 AI-Generated Security Recommendations

Security recommendations section listing 5 actionable steps including avoiding links, verifying through official channels, and reporting phishing.



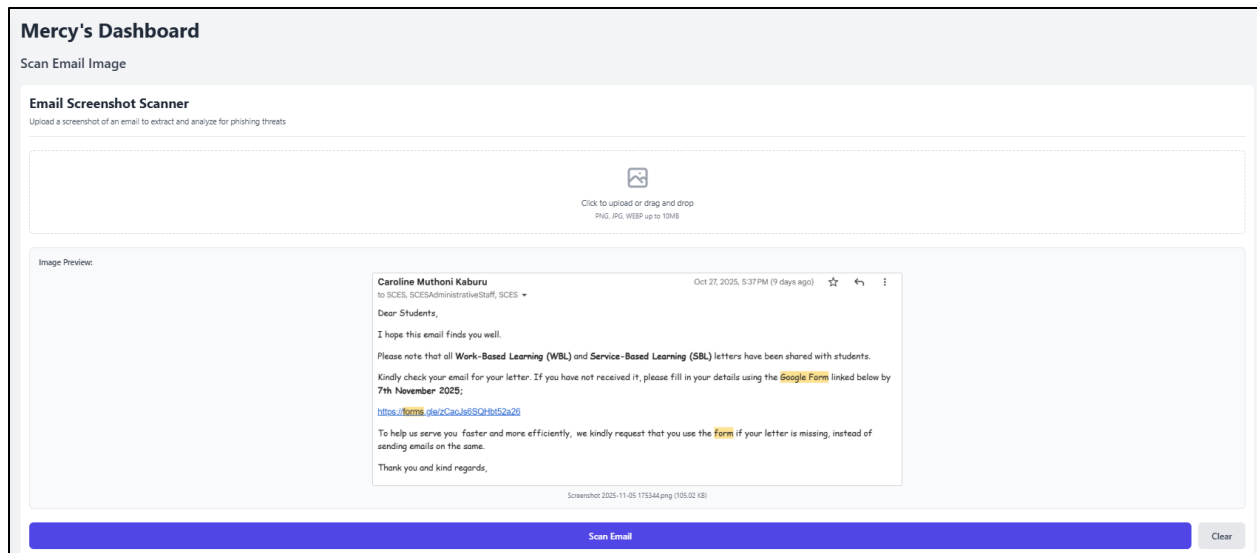
Appendix 16 Confidence Scoring Dashboard

Three confidence indicators displayed with visual meters showing OCR (96%), Email (High%), and Phishing (98%) confidence levels.

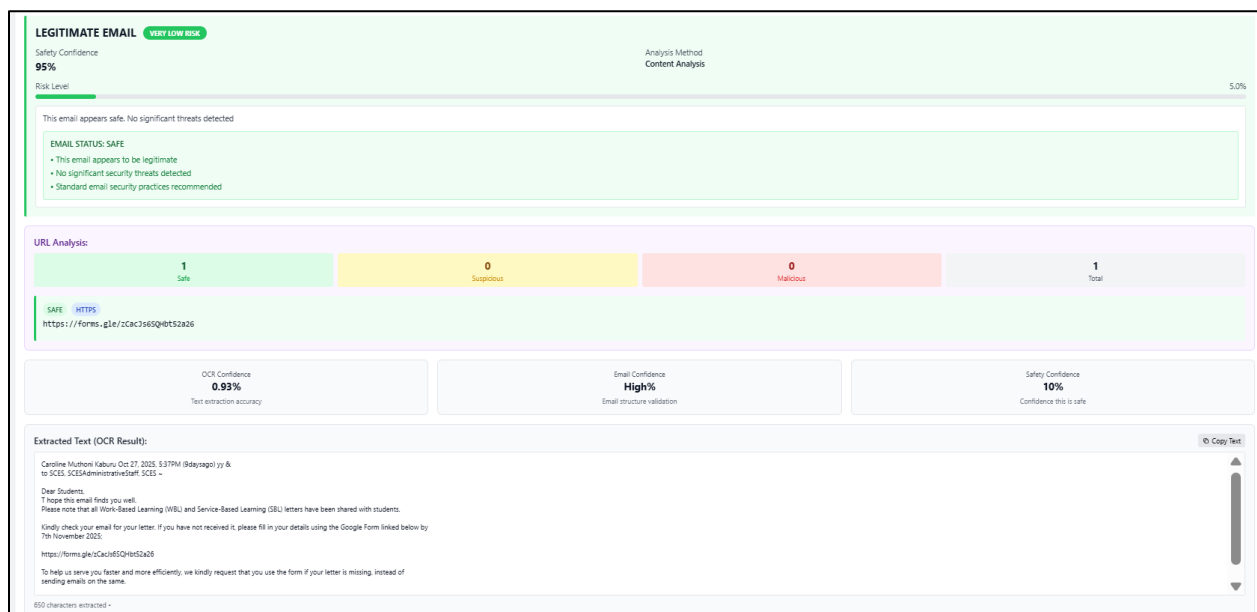


Appendix 17 Comprehensive AI Analysis Dashboard

Full-page screenshot showing complete integrated dashboard with all components: phishing detection alert at top, 98% confidence gauge, risk level indicator, detailed risk factors list, URL analysis section, security recommendations panel, confidence metrics (OCR 96%, Email High%, Phishing 98%), and extracted text display at bottom.

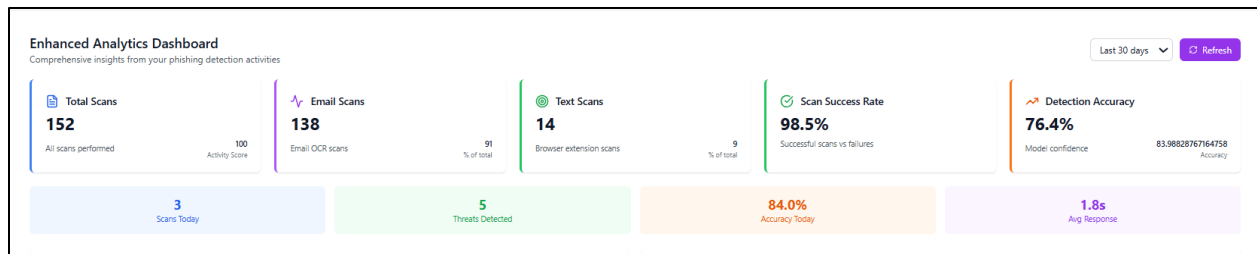


Appendix 18 Legitimate Email Screenshot



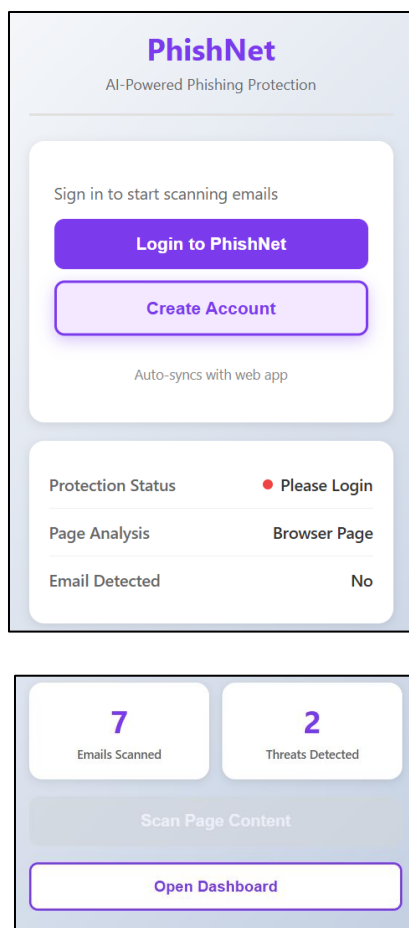
Appendix 19 Model Validating Legitimate Email

Dashboard showing legitimate email screenshot in Appendix 18 and the legitimate classified as as "SAFE" with 95% safety confidence and green risk indicator with no suspicious patterns detected in Appendix 19.



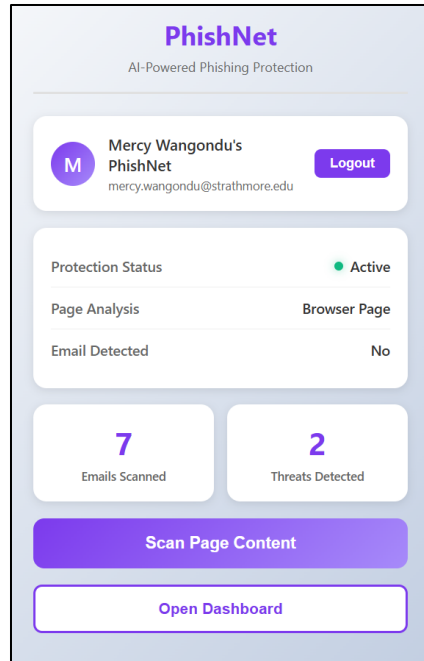
Appendix 20 Analysis Response Time Performance

User analytics dashboard showing average complete phishing analysis completed in 1.8 seconds.



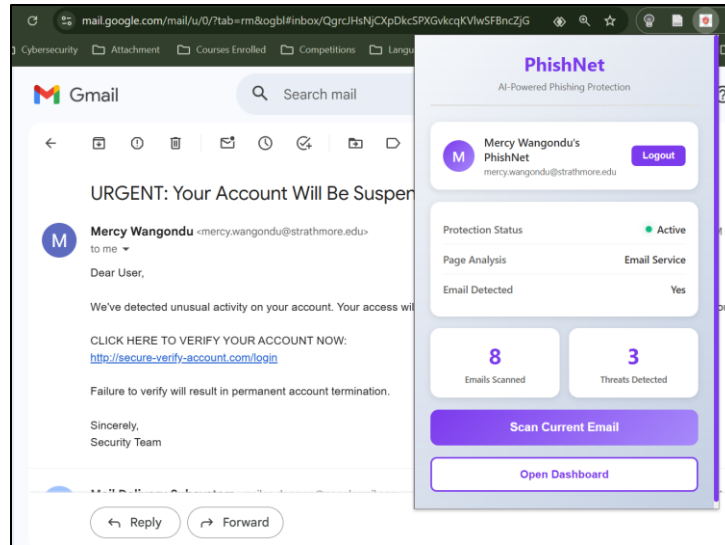
Appendix 21 PhishNet Extension Before Authentication

Extension popup showing "Login to PhishNet" button redirecting to login page (Appendix 4), "Create Account" button redirecting to registration page (Appendix 3), "Protection Status: Please Login" message, disabled scan features, and "Open Dashboard" button that redirects to login when clicked without authentication.



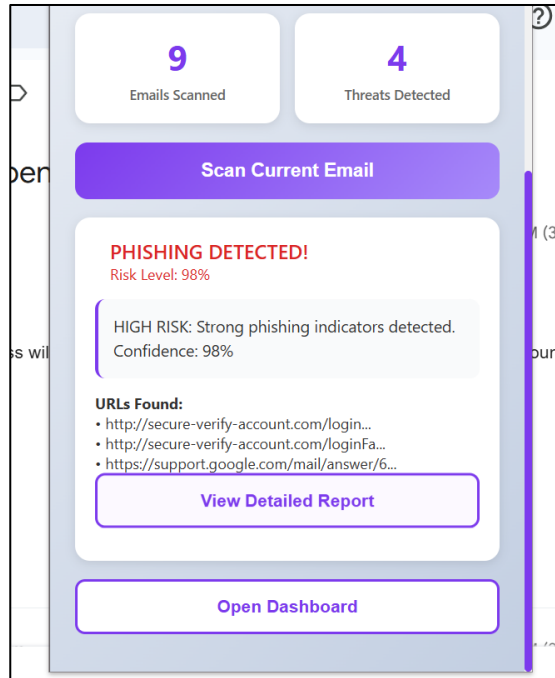
Appendix 22 PhishNet Extension After Successful Login

Extension displaying “Mercy Wangondou's PhishNet” with the email above , Protection Status: Active", enabled scan features, active statistics showing 8 Emails Scanned and 3 Threats Detected, logout button, and accessible "Open Dashboard" button.



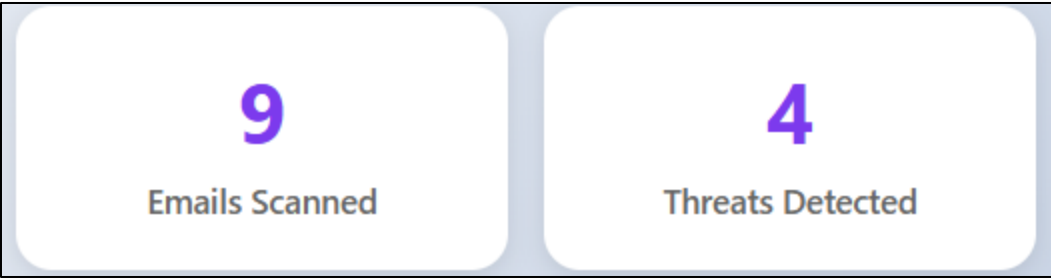
Appendix 23 Extension Detecting Email Page

Extension popup showing "Page Analysis: Email Service" and "Email Detected: Yes" with "Scan Current Email" button ready for analysis. Email detection status displays "Yes" only when an email is detected on the page and remains "No" on non-email pages. Background shows active email page confirming detection accuracy.



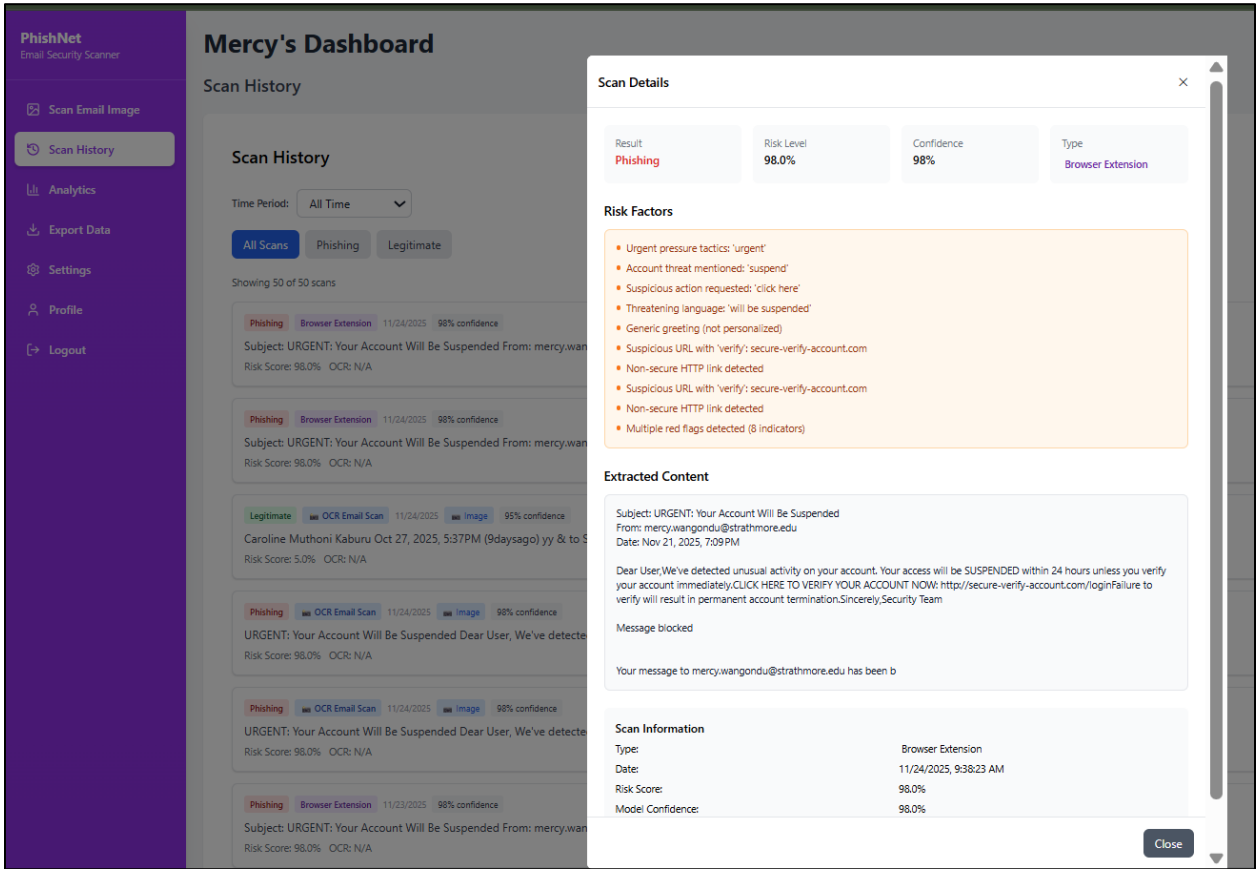
Appendix 24 Immediate Threat Detection Alert

Extension popup displaying "PHISHING DETECTED! Risk Level: 98%" with HIGH RISK warning, confidence score, detected URLs <http://secure-verify-account.com/login> , "View Detailed Report" button, and updated statistics showing 9 emails scanned and 4 threats detected. The analysis uses the same AI model demonstrated in Section 1.1.3 AI Model Testing and produces identical results for both phishing and legitimate emails (as shown in TC018). The only difference between extension scanning and web app analysis is the email acquisition method: the extension performs direct email content extraction while the web app uses OCR extraction from uploaded images.



Appendix 25 Real-Time Statistics Counter Update

Extension showing incremented counters after scan completion with "9 Emails Scanned" and "4 Threats Detected" displayed in popup interface, incremented from 8 emails and 3 threats as seen in Appendix 23.



Appendix 26 Complete Scan Report in Dashboard History Section

Dashboard history displaying comprehensive analysis report showing Result: Phishing, Risk Level: 98%, Confidence: 98%, Type: Browser Extension, 8 identified risk factors, extracted email

content, sender ,date , message body, and scan information including scan type, timestamp, risk score , and model confidence.